Deep Bayesian Inference for Control: Normalizing Flow – Based Approaches to System Identification, State Estimation, and Optimal Control

genehmigte Dissertation
zur Erlangung des akademischen Grades der Doktor-Ingenieurin (Dr.-Ing.).
an der
Fakultät für Angewandte Informatik
der
Universität Augsburg

vorgelegt von
Yi Zhang
aus
Tangshan, China

2026

Dekan: Prof. Dr. Jörg Hähner
Doktorvater: Prof. Dr.-Ing. Lars Mikelsons

I dedicate this thesis to my family.

Contents

Acronyms **i**

List of Figures **vi**

List of Tables **ix**

I Introduction **1**

1 Motivation 1

2 Background and Preliminaries 5

2.1 Notations 6

2.2 Bayesian Inference 6

2.2.1 Bayes' Rule 6

2.2.2 Markov Chain Monte Carlo 8

2.2.3 Variational Inference 10

2.3 Deep Probabilistic Methods 12

2.3.1 Deep Gaussian Process 12

2.3.2 Bayesian Neural Network 14

2.3.3 Variational Autoencoder 15

2.4 Normalizing Flow 17

2.4.1 Basics 17

2.4.2 Coupling Flows 20

2.4.3 Continuous Flows 23

2.4.4 Flow Matching 25

2.4.5 Conditional Normalizing Flows 26

2.5 From Bayesian Inference to Control 28

2.5.1 Bayesian System Identification 28

2.5.2 Bayesian State Estimation . . . 30
2.5.3 Model Predictive Control . . . 34
2.5.4 Reinforcement Learning for Control . . . 36
3 Objectives and Contributions . . . 38
4 Outline . . . 38

II Literature Review **41**
1 Bayesian Parameter Identification . . . 41
1.1 Deep Gaussian Processes . . . 41
1.2 Bayesian Neural Networks . . . 42
1.3 Conditional Variational Autoencoders . . . 42
1.4 Conditional Normalizing Flows . . . 43
1.5 Conditional Diffusion Models . . . 43
2 Bayesian State Estimation . . . 43
2.1 Data-Assisted Bayesian Filters . . . 44
2.2 Deep State-Space Models . . . 44
3 Deep Probabilistic Models and Time Series Modeling . . . 45
3.1 Integration of Deep Probabilistic Models with Time Series Modeling Architectures . . . 45
3.2 Normalizing Flows and Stochastic Processes . . . 46
4 Deep Generative Models as Control Policy . . . 47

III Methodologies **51**
1 Parameter Identification . . . 51
1.1 Universal Framework with Deep Generative Models for Amortized Bayesian Inference . . . 51
1.2 Normalizing Flows for Parameter Identification . . . 52
1.3 Stochastic Normalizing Flows . . . 53
1.4 Relationship Between Sensitivity Analysis . . . 55
1.5 When Does the Framework Fail? . . . 55
2 State Estimation . . . 56
3 Optimal Control . . . 57

IV Experiments and Results **59**
1 Overview . . . 59
2 Parameter Identification . . . 59
3 State Estimation . . . 59
4 Optimal Control . . . 59

V Discussion and Conclusion **61**

Abstract **63**

Zusammenfassung **65**

Bibliography **66**

Own Contributions and Publications **67**
Own share in data acquisition and data analysis 67
Own Publications 67

Appendix **89**

Acknowledgement **91**

Eidesstattliche Versicherung (Affidavit) in German **93**

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel **95**

Acronyms

DGM Deep Generative Model

VAE Variational Autoencoder

CVAE Conditional Variational Autoencoder

NF Normalizing Flow

CNF Conditional Normalizing Flow

SNF Stochastic Normalizing Flow

FM Flow Matching

DDPM Denoising Diffusion Probabilistic Models

BNN Bayesian Neural Network

RNN Recurrent Neural Network

LSTM Long Short-Term Memory

NARX Nonlinear Autoregressive eXogenous

MPC Model Predictive Control

DGP Deep Gaussian Process

PDF Probability Density Function

CDF Cumulative Distribution Function

MCMC Markov Chain Monte Carlo

MH Metropolis-Hastings

MALA Metropolis-adjusted Langevin algorithm

HMC Hamiltonian Monte Carlo

ELBO Evidence Lower Bound

KL Kullback-Leibler

VI Variational Inference

GPR Gaussian process regression

SE squared exponential

RBF Radial Basis Function

GP Gaussian Process

DGP Deep Gaussian Process

SGD Stochastic Gradient Descent

CAVI Coordinate Ascent Variational Inference

BbB Bayes by Backprop

CNN Convolutional Neural Network

RNN Recurrent Neural Network

ODE ordinary differential equation

SDE stochastic differential equation

NICE Non-linear Independent Component Estimation

RealNVP Real-Valued Non-Volume Preserving

MAF Masked Autoregressive Flow

i.i.d. independent and identically distributed

INN Invertible Neural Network

cINN conditional Invertible Neural Network

HINT Hierarchical Invertible Neural Transport

MPC Model Predictive Control

DIC Deviance Information Criterion

MAP maximum a posteriori

KF Kalman Filter

EKF Extended Kalman Filter

UKF Unscented Kalman Filter

PF Particle Filter

PID Proportional-Integral-Derivative

RL Reinforcement Learning

MDP Markov Decision Process

PINN Physics-Informed Neural Network

PDE Partial Differential Equation

SSM State-Space Model

EM Expectation-Maximization

SDE Stochastic Differential Equation

SCNF Stochastic Continuous Normalizing Flow

CTFP Continuous Time Flow Process

List of Figures

I.1 Knowledge Map 6
I.2 Variational Autoencoder Architecture 16
I.3 Coupling Flow Architecture 21
I.4 BayesFlow Architecture 27
I.5 Model Predictive Control (MPC) and Reinforcement Learning (RL). 34

III.1 Universal Framework of Using Deep Generative Models for Amortized Bayesian Inference 52
III.2 SNF Architecture 53

List of Tables

I.1 Notation Summary 7

II.1 **Normalizing Flows in Reinforcement Learning** 47

Chapter I

Introduction

1 Motivation

In this seemingly certain world, uncertainty lingers like a shadow, yet from the uncertain, we often find certainty.

The paradoxical relationship between certainty and uncertainty manifests across disciplines, demonstrating how truth often arises from navigating ambiguity. Quantum mechanics redefines predictability under randomness in the subatomic world, where Newtonian deterministic physics collapses. This uncertainty led to groundbreaking laws (e.g. Heisenberg's Uncertainty Principle), which now underpin modern technologies like quantum computing and all forms of spectroscopy. An example of medical diagnosis is that a patient presents vague symptoms, such as fatigue or pain, which could indicate anything from stress to cancer. Doctors converge on the cause through iterative hypothesis-testing approaches. Climate science also epitomizes how we manage uncertainty. Climate models incorporate a wide range of sources of variability and uncertainty. A common way to address it is to combine the results of a collection of different simulation models. By running multi-model ensembles, scientists can identify consistent patterns across the uncertainty.

From quantum randomness to medical diagnosis to climate models, uncertainty does not destroy the deterministic systems, but provides us accessibility to identify the systems. This perspective supports the use of probabilistic inference methods, which offer a rigorous framework for representing uncertainty and reasoning from uncertainty. In statistics, modeling uncertainty of variables from given data samples, using a probability distribution rather than just a point estimate is known as inference *. The estimated distribution facilitates uncertainty quantification and provides a measure of reliability of the results, making it particularly well-suited for real-world applications.

Probabilistic inference grounded in Bayes' rule is known as ***Bayesian inference***.

*We note that "inference" is an overloaded term in this context. We distinguish between: (i) Bayesian Inference, the statistical goal of recovering the posterior distribution; (ii) Variational Inference, the mechanism to achieve Bayesian inference; (iii) Inference: the process that maps data to latent space in generative models, which is in constrast to generation; and (iv) Computational Inference, the deep learning engineering practice of executing the generation pass (from latent to data) of a trained network on test data.

Bayes' rule interprets unknown random variables with beliefs (the prior). The beliefs are updated using observed data (likelihood), yielding the posterior distribution. A simplified formulation is:

$$\text{posterior} = \text{prior} \times \text{likelihood} \,/\, \text{total evidence (or marginal likelihood)}.$$

For example, we can illustrate Bayes' rule using the medical diagnosis scenario: the objective is to obtain the probability of a disease given a positive test result:

$$P(\text{Disease}|\text{Positive}) = \frac{P(\text{Positive}|\text{Disease}) \times P(\text{Disease})}{P(\text{Positive})},$$
$$P(\text{Positive}) = P(\text{Positive}|\text{Disease}) \times P(\text{Disease}) + P(\text{Positive}|\overline{\text{Disease}}) \times P(\overline{\text{Disease}}),$$

where $\overline{\text{Disease}}$ denotes "No Disease". In a more general Bayesian context, inference is the process of obtaining the posterior distribution of target variables from observed data. This perspective extends naturally to modern machine learning, where unknown values, structures, and policies can all be understood as random variables and probability distributions (Murphy, 2022).

Especially in modern Deep Generative Models (DGMs), Bayesian inference plays an essential role (Murphy, 2023). Through introducing random variables to represent complex data in latent space, DGMs enable data synthesis from latent variables (generation) based on the recognition of underlying latent representation from data (inference). However, directly inferring the posterior distribution over the latent variables using Bayes' rule I.2 is generally intractable, since all possible values of the latent variables are rarely obtainable in complex and high-dimensional models. Addressing this challenge is essential to fully unfold the potential of DGMs. Variational Autoencoders (VAEs) (Kingma and Welling, 2019) tackle this problem by introducing an inference network as a tractable approximation to the posterior. Normalizing Flows (NFs) (Rezende and Mohamed, 2016) construct bijective transformations between observed data and latent variables, achieving exact computation of posteriors. Diffusion models (Ho et al., 2020) sidestep calculating the posterior distribution over latent variables by designing it as a simple Gaussian, and instead frame both inference and generation as an iterative denoising process from noise through learned transformations.

To sum up, DGMs are inherently probabilistic; through probabilistic modeling and inference, they are powerful in extracting latent information and representing uncertainty in complex data. These capabilities are proven in numerous applications in images, text, and speech. The same principles extend naturally to scientific and engineering domains. One prominent example is: how can DGMs empower identification and modeling of complex, stochastic, nonlinear dynamical systems? In control theory, the process of building mathematical models of dynamical systems from observed input-output data is

called ***system identification***. Interpreting system identification in probabilistic terms, dynamics modeling is inferring the posterior of the next states based on the previous states and system inputs, while parameter identification is estimating the posterior over system parameters given observations. Identifying system in probabilistic way enables uncertainty quantification, which is crucial for reliable and robust decision-making in real-world applications.

Classic methods for system identification rely on linearized models or pre-specified nonlinear structures. Model parameters are then estimated from observed data using estimation techniques. Subsequently, a validation process is employed to assess and select the most appropriate model structure. Although these methods offer strong physical interpretability and are often data-efficient, they exhibit several limitations: parameter estimation can be computationally inefficient with large amounts of data; results are sensitive to the choice of model structures and dependent on expert knowledge; uncertainty is difficult to handle and quantify; and they are generally inapplicable to partially observable systems and high-dimensional and raw data.

Despite the demand for uncertainty quantification, deep learning methods are suitable for system identification thanks to their strength in representing information from complex data, flexibility of function approximation, scalability in high-dimensional data, and compatibility with optimization techniques. Taking uncertainty quantification into account, a simple yet effective technique is Gaussian Process Regression (Särkkä, 2019), which involves assigning a Gaussian process prior to the latent regressor and performing statistical conditioning based on the available observations. This method has gained widespread adoption due to its non-parametric characteristic and adaptability. Nonetheless, its significant computational and memory requirements limit its feasibility when dealing with large datasets. Another straightforward approach to combining the advantages of deep learning methods and the possibility of uncertainty modeling is the Bayesian Neural Network (BNN) (Goan and Fookes, 2020). Unlike standard neural networks that produce point estimates, BNNs assign probability distributions over the network parameters, thereby inducing distributions for outputs. However, BNNs reveal expressiveness limitations because the approximate inference may not reliably capture true posterior behavior (Foong et al., 2020). Therefore, the adaptation and application of DGMs are essential for achieving rich representational capacity, flexible modeling frameworks, and the ability to effectively handle large-scale data.

Different DGMs have been investigated in system identification applications. VAEs are flexible to cooperate with time series modeling architectures such as Recurrent Neural Network (RNN) (Girin et al., 2021) and Nonlinear Autoregressive eXogenous (NARX) (Paniagua and López, 2024), which is beneficial in uncovering the time-dependent features from data. However, VAEs suffers from a well-known problem of shrinkage of variance

(Asperti, 2020) and tends to converge to a degenerate local optimum known as posterior collapse (Wang et al., 2023). Unlike VAEs, NFs are restricted in their architectural choices due to the requirement of invertibility, which limits their direct applicability to time series modeling. On the other side, NFs provide exact and expressive density estimation capabilities, qualifying them for capturing uncertainty in data and enhancing the representational capacity of probabilistic forecasts. Diffusion models release the architectural constraints and manifest the strength in generating high-quality and coherent predictions. Nevertheless, their iterative data generation procedure incurs substantial computational overhead. Balancing expressiveness, computational efficiency, and sample quality, ***normalizing flows*** are chosen in this work as the generative backbone of probabilistic system identification.

Based on the results of system identification, ***state estimation*** is the process of inferring the value of hidden, time-varying state variables of a dynamical system from a history of noisy and potentially partial observations and known control inputs. Because of the noise in the sensors (measurement noise) and unmodeled disturbances in the system itself (process noise), state estimation is naturally probabilistic. Thus, estimating the target states is computing the posterior distribution of target states. The traditional method for state estimation is the Luenberger State Observer, which assumes the systems are deterministic and both the system dynamics and measurement models are linear and time-invariant. Taking stochasticity into account, Kalman Filter is the most widely applied method. Beyond system linearity, a Kalman Filter assumes the process and measurement noises are additive Gaussian noise with zero mean. In order to handle nonlinear systems, Extended Kalman Filter linearizes the nonlinear functions around the state estimate. However, the linearization introduces errors, leading to wrong estimation or even divergence. An improved modification is the Unscented Kalman Filter, which carefully selects the means and variances and propagates them through the nonlinear functions. However, it is still only compatible with Gaussian noises.

Through relaxing the need for precise modeling of dynamical systems and avoiding restrictive assumptions about uncertainty—while preserving expressiveness, modeling flexibility, and scalability to large datasets—the chosen architecture for probabilistic system identification, NF, also provides a foundation for accurate, efficient and reliable probabilistic state estimation. From a data-centric perspective, parameter identification involves mapping time series to discrete data points; state estimation entails inferring data points or reconstructing time series from temporal data. In order to address these tasks, it is necessary to extend normalizing flows, given the limited research currently available in this area. Bridging this gap not only broadens the applicability of NFs but also contributes to advancing Bayesian inference techniques for complex temporal systems.

Having explored system identification and state estimation through Bayesian inference, we can advance toward ***optimal control*** by leveraging data as a central source of insight

into system behavior, enabling more adaptive, scalable, and robust control strategies. This is especially advantageous in situations where analytical models are incomplete, uncertain, or impractical to derive. One unmediated application is model-free control, which obtains control inputs directly from observations and reference trajectories without requiring an explicit system model. This enables fast and flexible path planning and decision-making, particularly when multiple samples or rollouts are required for real-time path planning and tracking tasks. Building upon model-free principles, Reinforcement Learning (RL) extends this paradigm by optimizing control policies through interaction with the environment. RL agents learn to maximize long-term rewards while adapting to system uncertainties and disturbances, effectively extend the capability of traditional Model Predictive Control (MPC). By combining Bayesian inference with data-driven exploration, RL offers a powerful framework for achieving uncertainty-aware, robust, and self-improving control in dynamic environments.

In summary, this work is motivated by three key considerations:

- Bayesian inference provides a principled framework for modeling uncertainty and extracting knowledge from uncertainty, enabling system identification and state estimation based solely on observations. This is particularly valuable in scenarios where first-principles models are unavailable, incomplete, or prohibitively complex.

- System identification and state estimation are required to be expressive for representing complex models, capable of modeling uncertainty, and scalable to large data sets. Deep Generative Models (DGMs), in particular Normalizing Flows (NFs), fulfill these criteria. However, existing architectures must be further developed to adapt to sequential data more flexibly.

- By leveraging Bayesian inference to learn control policies and refining them through reinforcement learning, model-free control enables adaptive, uncertainty-aware decision-making in complex, unpredictable environments. This approach has not been fully explored and offers significant potential for advancing control strategies.

2 Background and Preliminaries

This section introduces the necessary foundational concepts and methodologies for understanding the rest of this work. Before the main contents, we explain the unified notations applied in this whole work. We first review classical Bayesian inference methods. Then we introduce deep probabilistic methods, including Deep Gaussian Process (DGP), Bayesian Neural Network (BNN), and Variational Autoencoder (VAE). Next, we present the core methods of this work, Normalizing Flows (NFs), a powerful class of generative models

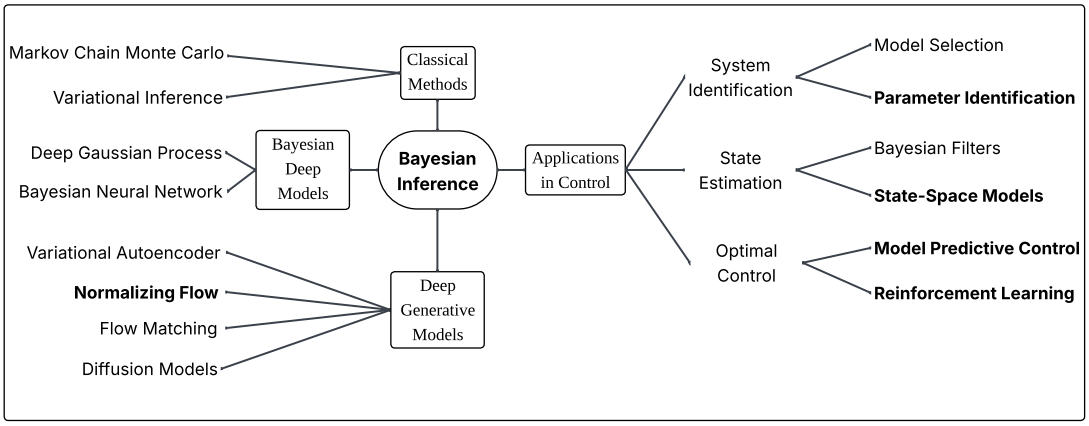


Figure I.1: Knowledge Map of Foundational Theories

for flexible density estimation. Finally, we bridge Bayesian inference to control methods, covering system identification, state estimation, and model predictive control. The overview for this chapter is shown in Figure I.1.

2.1 Notations

To maintain clarity throughout this document, we adopt the following notation:

2.2 Bayesian Inference

Bayesian inference gets its name from using Bayes' rule to perform statistical inference of probability distributions. This chapter refers to *Bayesian Data Analysis* (Gelman et al., 2013), *Handbook of MCMC* (Allen, 2007), and *Variational Inference* (Blei et al., 2017).

2.2.1 Bayes' Rule

Bayes' rule is derived from the chain rule of conditional probability. For target variables $\boldsymbol{\theta}$ and the corresponding observed data $\mathcal{D}$, the chain rule states that:

$$p(\boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\theta}|\mathcal{D})p(\mathcal{D}) = p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{I.1}$$

Rearranging the terms gives Bayes' rule or Bayes' theorem:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}. \tag{I.2}$$

Category	Description
Basic Symbols	Scalars: x; Vectors: $\boldsymbol{x}$; Matrices: $\boldsymbol{X}$; Sets: $\mathcal{D}$
Random Variables	Multidimensional random variables: $\boldsymbol{x} = [x_1, \ldots, x_N]^T$; Sample index: $\boldsymbol{x}^{(i)}$; Stochastic processes: $X(t)$
Time Series	Time series of length T: $\boldsymbol{x}_{(1:T)}$; Time-dependent variables: $\boldsymbol{x}(t)$ or $x(t)$; Dimension index: $\boldsymbol{x}_i$
Variable Modifications	Estimated: $\hat{\boldsymbol{x}}$; Transformed: $\tilde{\boldsymbol{x}}$; Time derivative: $\dot{\boldsymbol{x}}$
Probability and Statistics	PDF: $p(\cdot)$; Joint PDF: $p(\cdot, \cdot)$; Conditional PDF: $p(\cdot \mid \cdot)$; Expectation: $\mathbb{E}_{p(x)}[f(x)]$; Covariance: $\mathrm{Cov}[\boldsymbol{x}, \boldsymbol{y}]$; Variance: $\mathrm{Var}[\boldsymbol{x}]$
Matrix Operations	Identity: $\boldsymbol{I}$; Jacobian: $\boldsymbol{J}$; Determinant: $\det \lvert\boldsymbol{J}\rvert$; Trace: $\mathrm{Tr}(\boldsymbol{X})$; Transpose: $\boldsymbol{X}^\top$

Table I.1: Notation Summary

In Bayes' theorem, $p(\boldsymbol{\theta})$ is the initial distribution of the variables before any data samples are considered, which is called the prior distribution. $p(\boldsymbol{\theta}|\mathcal{D})$ is the posterior distribution of the target parameter after being updated by the observed data. $p(\mathcal{D}|\boldsymbol{\theta})$ is the likelihood of expected data given the setting of parameters $\boldsymbol{\theta}$. $p(\mathcal{D})$ is called the marginal likelihood or evidence of expected data across all possible parameter settings:

$$p(\mathcal{D}) = \Sigma_{\boldsymbol{\theta}'} p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}', \tag{I.3a}$$

$$p(\mathcal{D}) = \int p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')d\boldsymbol{\theta}'. \tag{I.3b}$$

In practice, it is often treated as a normalizing constant and ignored. The unnormalized posterior is then given by:

$$p_u(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{I.4}$$

Normally, the posterior distribution does not have a closed-form expression. Instead, we approximate it using numerical methods. Computational algorithms that rely on repeated random sampling to obtain numerical results refers to "Monte Carlo" methods. Monte

Carlo methods provide ways to approximate the posterior distribution by expectations:

$$E = \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}, \tag{I.5}$$

where $f(\boldsymbol{\theta})$ is any function of interest, for example, $f(\boldsymbol{\theta}) = \boldsymbol{\theta}$. As the simplest way, we can approximate it using standard numerical techniques such as a Riemann sum over a discrete grid of points:

$$E \approx \Sigma_{\boldsymbol{\theta}'} f(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}'. \tag{I.6}$$

However, this approach is only feasible for low-dimensional $\boldsymbol{\theta}$ due to the curse of dimensionality. Besides, for complex distributions, the discretization is highly inefficient. Therefore, for high-dimensional parameters and complex distributions, we often resort to sampling-based or optimization-based methods to approximate the posterior distribution, which will be discussed in the following sections.

2.2.2 Markov Chain Monte Carlo

The "Markov Chain" aspect indicates that these algorithms generate a sequence of samples where each sample depends only on the previous one. In contrast to the basic Monte Carlo methods, Markov Chain Monte Carlo (MCMC) methods produce auto-correlated samples, so that each new sample adds less new information than an independent draw would. Different sequences updating algorithms lead to different MCMC methods. The goal of MCMC is to construct a Markov chain that has the desired posterior distribution as its equilibrium distribution. After running the chain for a sufficient number of iterations, the samples generated from the chain can be treated as samples from the target posterior distribution.

Metropolis-Hastings (MH) algorithm

The MH algorithm, which is named after Metropolis (1953) and Hastings (1970), is a widely used MCMC method. The steps of the MH algorithm are as follows:

1. Initialize the chain with a starting point $\boldsymbol{\theta}_{(0)}$, choose a proposal distribution $q(\boldsymbol{\theta}'|\boldsymbol{\theta})$.
2. For each iteration i:
   (a) Propose a new state $\boldsymbol{\theta}'_{(i)}$ from the proposal distribution $q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)})$.
   (b) Calculate the transition probability, also known as the acceptance ratio:

$$T(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) = \min\{1, \frac{p(\boldsymbol{\theta}'_{(i)}|\mathcal{D})q(\boldsymbol{\theta}_{(i-1)}|\boldsymbol{\theta}'_{(i)})}{p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)})}\}. \tag{I.7}$$

(c) Generate a uniform random number $u \sim U(0,1)$. Update $\boldsymbol{\theta}_{(i)}$ by "accept or reject":

$$\boldsymbol{\theta}_{(i)} = \begin{cases} \boldsymbol{\theta}'_{(i)}, & \text{if } u < T(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \\ \boldsymbol{\theta}_{(i-1)}, & \text{otherwise} \end{cases}. \tag{I.8}$$

3. Repeat step 2 for numerous iterations to obtain a sequence of samples $\{\boldsymbol{\theta}_{(1)}, \ldots, \boldsymbol{\theta}_{(N)}\}$.

From the sequence of samples, we can construct a Monte Carlo estimator:

$$p(\boldsymbol{\theta}|\mathcal{D}) \approx \frac{1}{N}\Sigma_{i=1}^{N}\delta_{\boldsymbol{\theta}_{(n)}}(\boldsymbol{\theta}). \tag{I.9}$$

Although this procedure allows flexible approximation of complex posterior distribution, it suffers from slow convergence rate due to its random exploration nature. Thus, gradient-based MCMC methods are proposed to incorporate the structure of the target density, leading to faster sampling.

Metropolis-adjusted Langevin algorithm (MALA)

The MALA (Besag, 2004) combines the MH algorithm with Langevin diffusion, whose stationary distribution is the target density. In general, for a stochastic process $\Theta(t)$, the Langevin diffusion is described by the following Stochastic Differential Equation (SDE):

$$d\Theta(t) = \nabla \log p(\Theta(t))dt + \sqrt{2}dW(t), \tag{I.10}$$

where $W(t)$ denotes a standard Wiener process (Gabbiani and Cox, 2010). The Wiener process is a continuous-time stochastic process with independent, normally distributed increments, which can be interpreted as Brownian motion. It has the following properties:

(1) $W(0) = \mathbf{0}$;

(2) for $t > s \geq 0$, $W(t) - W(s) \sim \sqrt{t-s}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I})$;

(3) for $t_1 > s_1 \geq 0$, and $t_2 > s_2 \geq 0$, $W(t_1) - W(s_1)$ and $W(t_2) - W(s_2)$ are independent.

The MALA algorithm uses the first order Euler-Maruyama discretization of the SDE, resulting in the following proposal distribution:

$$\Theta(t) = \Theta(i-1) + \tau\nabla \log p(\Theta(i-1)) + \sqrt{2\tau}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{I.11}$$

where τ is a step size parameter. Because of the discretization, convergence to the target density is no longer guaranteed. Therefore, the MH algorithm is applied to correct the

bias introduced by the discretization. For each iteration i, the MALA algorithm updates the state as follows:

(a) Compute the gradient of the log-posterior at the current state: $\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})$.

(b) Propose a new state $\boldsymbol{\theta}'_{(i)}$ using the Langevin dynamics (I.11).

(c) Calculate the transition probability according to Equation I.7. Specifically, the proposal distribution is derived from the Langevin dynamics:

$$q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \propto \exp(-\frac{1}{4\tau}\left\|\boldsymbol{\theta}'_{(i)} - \boldsymbol{\theta}_{(i-1)} - \tau\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})\right\|_2^2). \tag{I.12}$$

(d) (same as MCMC) Generate a uniform random number $u \sim U(0,1)$. Update $\boldsymbol{\theta}_{(i)}$ by "accept or reject" according to Equation I.8.

Compared to the MH, MALA converges faster and scales better in higher dimensions, at the cost of requiring gradients and tuning. As a more advanced MCMC method, Hamiltonian Monte Carlo (HMC) (Duane et al., 1987) further improves sampling efficiency by simulating the whole sampling trajectory as Hamiltonian dynamics.

2.2.3 Variational Inference

Different from MCMC, Variational Inference (VI) (Jordan et al., 1998; Wainwright and Jordan, 2008) is an optimization-based method that approximates the target posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ with a simpler, parameterized distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$, where $\boldsymbol{\phi}$ are the variational factors. The goal of VI is to find the optimal variational factors $\boldsymbol{\phi}^*$ that minimize the distance between the approximate distribution and the true posterior. The distance is normally measured by the Kullback-Leibler (KL)-divergence (Kullback and Leibler, 1951), which is defined as:

$$D_{\mathrm{KL}}(q(z)||p(z)) = \mathbb{E}_{q(z)}[\log \frac{q(z)}{p(z)}] = \int q(z) \log \frac{q(z)}{p(z)} dz. \tag{I.13}$$

Thus, the optimization problem of VI can be formulated as:

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})). \tag{I.14}$$

Substituting Bayes' rule (I.2) into KL-divergence, we have:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||\frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}). \tag{I.15}$$

The marginal likelihood $p(\mathcal{D})$ is usually intractable, as it involves integrating over all possible parameter settings. Therefore, the divergence cannot be computed directly.

Expanding the above equation with the definition of KL-divergence, we have:

$$\begin{aligned} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})}[\log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \log p(\mathcal{D}|\boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) + \log p(\mathcal{D})] \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}). \end{aligned} \tag{I.16}$$

Since $p(\mathcal{D})$ is independent of the variational factors $\boldsymbol{\phi}$, the last term is reduced to $\log p(\mathcal{D})$. Combining the terms $p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = p(\boldsymbol{\theta}, \mathcal{D})$ gives

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) + \log p(\mathcal{D}) \tag{I.17}$$

Or combining the terms $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$ and $p(\boldsymbol{\theta})$ via KL-divergence gives:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\mathcal{D}) \tag{I.18}$$

Rearranging the terms in Equation I.17 gives:

$$\log p(\mathcal{D}) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{I.19}$$

Since the KL-divergence is always non-negative, we have:

$$\log p(\mathcal{D}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{I.20}$$

Because of this inequality, the right hind side is called the Evidence Lower Bound (ELBO), also known as the variational lower bound. Another way to formulate the ELBO using Equation I.18 is:

$$\mathrm{ELBO} = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})). \tag{I.21}$$

Then, minimizing the KL-divergence is equivalent to maximizing the ELBO (I.20), leading to the equivalent optimization problem:

$$\boldsymbol{\phi}^* = \arg\max_{\boldsymbol{\phi}} \mathrm{ELBO} = \arg\min_{\boldsymbol{\phi}} [\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D})]. \tag{I.22}$$

To complete the optimization, we need to choose a suitable family of variational distributions $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$. A common choice is the mean-field family (Bishop, 2006), which assumes that the variables $\boldsymbol{\theta}$ are mutually independent and each governed by a distinct factor under q:

$$q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \Pi_i q_{\phi_i}(\theta_i), \tag{I.23}$$

where each factor ϕ_i governs a variable θ_i. Then, the ELBO (I.20) is iteratively optimized by updating each factor while holding the others fixed. The solution of the optimal distribution $q_{\phi_i}(\theta_i)$ is proportional to the exponential expected log of the complete condition:

$$\log q^*_{\phi_i}(\theta_i) \propto \mathbb{E}_{\Pi_{j\neq i} q_{\phi_i}(\theta_i)}[\log p(\theta_i|\boldsymbol{\theta}_{j\neq i}, \mathcal{D})], \tag{I.24}$$

where the expectation is taken with respect to the distributions of all other groups $j \neq i$. This algorithm is called Coordinate Ascent Variational Inference (CAVI) (Bishop, 2006).

The assumption of the mean-field VI simplifies the optimization problem, and thus makes it scalable for large dimensional variables and also enables parallelizable computation. However, it may also limit the expressiveness of the variational distribution, especially when the posterior correlations are crucial, leading to suboptimal approximations of the true posterior. Summarizing, VI is generally faster than MCMC and scales better to large datasets, but it may provide less accurate approximations of the posterior distribution (Blei et al., 2017).

2.3 Deep Probabilistic Methods

The classical methods, MCMC and VI, although provide general solution for Bayesian inference, they are limited by their "approximation-per-point" behavior. Deep probabilistic methods, on the contrary, enable amortized Bayesian inference and unlock both high expressiveness and scalability to large datasets. In the following section, we briefly review three prominent deep probabilistic methods: the Deep Gaussian Process, the Bayesian Neural Network, and the Variational Autoencoder. The former two methods are direct extensions of classical Bayesian inference with deep architectures, while the last one is a fundamental deep generative model that leverages variational inference.

2.3.1 Deep Gaussian Process

A Gaussian process can be interpreted as an infinite-dimensional Gaussian distribution, where at each time point the random variables follow normal distributions. For an N-dimensional random variable $\boldsymbol{x} = [x_1, \ldots, x_N]^T$, a Gaussian process $X(t)$ is fully specified by its mean function $\mu(\cdot)$ and covariance function $k(\cdot, \cdot)$:

$$\begin{bmatrix} X_1(t) \\ \cdots \\ X_N(t) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu(x_1) \\ \cdots \\ \mu(x_N) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1), \ldots, k(x_1, x_N) \\ \cdots \\ k(x_N, x_1), \ldots, k(x_N, x_N) \end{bmatrix} \right) \tag{I.25}$$

Using a Gaussian process to infer variables from observed data based on prior distributions leads to Gaussian process regression (GPR) (Rasmussen and Williams, 2008). For a

regression task with training data $\boldsymbol{x}$ and the ground truth $\boldsymbol{y}$, the GPR model is employed by a latent function $f(\cdot)$:

$$y_i = f(x_i) + \epsilon_i, \quad i = 1, \ldots, N, \tag{I.26}$$

where $\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$ are independent and identically distributed (i.i.d.) noises for all dimensions. We denote the collection of latent functions as $\boldsymbol{f} = [f(x_1), \ldots, f(x_N)]^T$. So, the likelihood of $\boldsymbol{y}$ is given by:

$$p(\boldsymbol{y}|\boldsymbol{f}) = \prod_{i=1}^{N} p(y_i|f(x_i)) = \prod_{i=1}^{N} \mathcal{N}(y_i|f(x_i), \sigma_\epsilon^2). \tag{I.27}$$

The prior distribution over functions $f(\cdot)$ is assumed to a zero-mean Gaussian process. We denote it as:

$$f(\cdot) \sim \mathcal{GP}(0, k(\cdot, \cdot)). \tag{I.28}$$

The kernel function $k(\cdot, \cdot)$ is usually chosen to be the squared exponential (SE) kernel function, or as known as the Radial Basis Function (RBF), defined as:

$$k(x_i, x_j) = \sigma_{SE}^2 \exp(-\frac{1}{2\ell^2} \|x_i - x_j\|^2), \tag{I.29}$$

where σ_{SE} and ℓ are vertical and horizontal scale parameters, which control the vertical span and smoothness of the function, respectively. Then, the posterior distribution of $\boldsymbol{y}$ can be computed analytically using the marginal likelihood over $\boldsymbol{y}$:

$$p(\boldsymbol{y}|\boldsymbol{x}) = \int \prod_{i=1}^{N} p(y_i|f(x_i)) p(f(x_i)|x_i) d\boldsymbol{f} = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K} + \sigma_\epsilon^2 \mathbb{I}), \quad \boldsymbol{K} = k(\boldsymbol{x}, \boldsymbol{x}). \tag{I.30}$$

The hyperparameters $\boldsymbol{\phi} = \{\sigma_{SE}, \ell, \sigma_\epsilon\}$ can be optimized by maximizing the log marginal likelihood $\log p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\phi})$.

The GPR is widely used for regression tasks owing to its non-parametric nature and its capacity to quantify uncertainty. However, its computational complexity scales cubically with the number of training points, i.e., $\mathcal{O}(N^3)$ for an N-dimensional input. Additionally, calculating the Gaussian processes requires the entire storage of training datasets, incurring a memory cost of $\mathcal{O}(M^2)$, which grows quadratically with the dataset size M. These limitations hinder its scalability to high-dimensional inputs and large datasets. In addition, the expressiveness of GPR is constrained by the choice of kernel function and its incapacity to perform hierarchical feature extraction.

To improve expressiveness, DGP (Damianou and Lawrence, 2013) is proposed as multilayer hierarchical generalizations of Gaussian processes. The input $\boldsymbol{x}$ is passed

through multiple layers of Gaussian processes to obtain the output $\boldsymbol{y}$. For an L-layer DGP, the model extends the Gaussian process (I.26) by:

$$y_i = f_L(f_{L-1}(\dots f_1(x_i) + \epsilon_{i,1} \dots)) + \epsilon_L, \quad f_l(\cdot) \sim \mathcal{GP}(0, k_l(\cdot,\cdot)), \\ l = 1, \dots, L,\ i = 1, \dots, n. \quad \text{(I.31)}$$

The deep architecture enables Gaussian processes to learn more complex data patterns and hierarchical representations. It can be seen as a deep neural network with infinitely wide layers (Neal, 1996), where each layer applies a Gaussian process, instead of nonlinear matrix transformation. However, due to the computational cost and storage consumption the DGP is still cumbersome to apply in practice.

2.3.2 Bayesian Neural Network

Another approach of deep probabilistic methods involves integrating Bayesian inference with deep neural networks. A Bayesian Neural Network (BNN) (Neal, 1996; MacKay, 1995) places prior distributions over the network's weights and biases, allowing uncertainty to be modeled directly within the architecture.

The goal of training a BNN is to infer the posterior distribution over the parameters $\boldsymbol{\theta} = [\boldsymbol{w}, \boldsymbol{b}]$ given the training data $\mathcal{D}$, i.e., $p(\boldsymbol{\theta}|\mathcal{D})$. VI is commonly employed to approximate the posterior by optimizing the ELBO (I.21). To facilitate gradient-based optimization via backpropagation—similar to standard neural networks—the reparameterization trick (Kingma and Welling, 2022) is used. This technique expresses random variables as deterministic variables of noise, enabling efficient and differentiable sampling during training. For example, for a Gaussian variational distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$, we can reparameterize $\boldsymbol{\theta}$ as:

$$\boldsymbol{\theta} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \quad \text{(I.32)}$$

where $\odot$ represents the Hadamard product, i.e., the element-wise product, and the variational parameters here are $\boldsymbol{\phi} = [\boldsymbol{\mu}, \boldsymbol{\sigma}]$. To train a BNN on large datasets, Stochastic Gradient Descent (SGD) (Bottou, 1999) is typically employed to optimize the ELBO. Unlike standard gradient descent, SGD estimates the gradient at each iteration using a randomly selected subset of the data. Then, the variational objective of a BNN using the ELBO (I.21) becomes:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \frac{1}{M} \sum_{i=1}^{M} [D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) || p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}_i|\boldsymbol{\theta})], \quad \text{(I.33)}$$

where each mini-batch $\mathcal{D}_i \subset \mathcal{D}$ is drawn from the equally sized subsets of the training

data $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_M\}$. During each optimization step, the auxiliary noise $\boldsymbol{\epsilon}$ is sampled multiple times from $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to perform Monte Carlo estimation of the expectation terms in the loss function $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$. The gradient $\nabla_{\boldsymbol{\phi}}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$ is then computed via backpropagation and averaged across all samples. Finally, the variational parameter $\boldsymbol{\phi}$ is updated by

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha\nabla_{\boldsymbol{\phi}}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}), \tag{I.34}$$

where α is the learning rate. This procedure is called Bayes by Backprop (BbB) (Blundell et al., 2015).

In comparison to DGP, the BNN demonstrates better scalability to large datasets and improved representational capacity. Moreover, the BbB method facilitates the compatibility of BNN with deep learning techniques and thus enables flexible implementation in different neural network architectures, such as Convolutional Neural Network (CNN) (Shridhar et al., 2019), RNN (Fortunato et al., 2019), and Transformer (Müller et al., 2024). Nevertheless, several works raise concerns with BNNs including wrong uncertainty estimation in between separated regions of observations (Foong et al., 2019, 2020), poor generalization ability under domain shift (Izmailov et al., 2021), and sensitive modeling ability to the choice of prior distributions (Silvestro and Andermann, 2020). These limitations motivate the applications of more powerful generative models, such as variational autoencoders and NFs, which are discussed in the following sections.

2.3.3 Variational Autoencoder

In classical VI methods, the parameters of interest $\boldsymbol{\theta}$ are random variables, and the approximate posterior distribution over the parameters is parameterized by variational parameters $\boldsymbol{\phi}$. In contrast, a Variational Autoencoder (VAE) is constructed with deterministic neural networks and introduces variability by latent variables. For a dataset $\mathcal{D} = \{\boldsymbol{x}^{(i)}\}_{i=1}^{N}$, the samples are assumed to be i.i.d. and drawn from an unchanging underlying distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$. A VAE extracts information from input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$ by a parameterized inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, which is also called an encoder or recognition model. Then, the VAE generates data $\boldsymbol{x}$ from the latent variables $\boldsymbol{z}$ by another parameterized model $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, which is also called a decoder. The decoder is often mentioned as a "generative" model. However, according to the formal definition (Kingma and Welling, 2019), a generative model learns a joint distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, including sampling $\boldsymbol{z} \sim p_{\boldsymbol{\theta}}(\boldsymbol{z})$ and sampling $\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$.

The latent variables connect the inference model and the generative model. From the encoder side, the inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ approximates the true posterior distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ of the generative model, which is intractable. From the decoder side, the distribution of latent variables is a prior distribution, which is usually chosen to be a standard normal

distribution, i.e., $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$. The whole architecture is illustrated in Figure I.2.

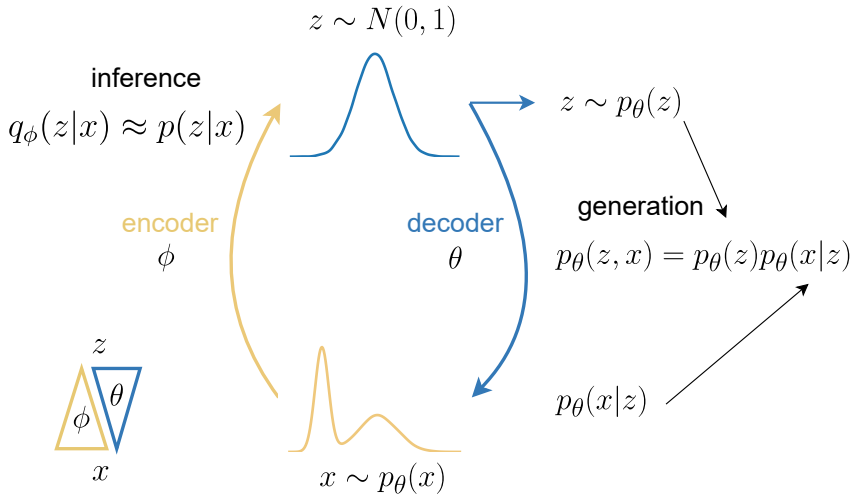


Figure I.2: The architecture of a Variational Autoencoder. The encoder (with neural network parameters $\boldsymbol{\phi}$) maps the input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$, and the decoder (with neural network parameters $\boldsymbol{\theta}$) reconstructs the input data from the latent variables.

Same reparameterization trick as in the BNN is used to enable gradient-based optimization. The latent variables $\boldsymbol{z}$ becomes:

$$\boldsymbol{z} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{I.35}$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x})$ and $\boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x})$ are also neural networks. Different from the mini-batch optimization in the BNN, we use one single encoder neural network to perform posterior inference over all samples in the dataset. This strategy of sharing variational parameters across datapoints is called amortized variational inference (Gershman and Goodman, 2014; Margossian and Blei, 2024), which improves the optimization efficiency.

The objective of the inference model is to minimize the KL-divergence between the inferred and true posterior $D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}))$. Recall the ELBO in VI (I.20, I.21), the ELBO of a VAE is given by:

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z})). \tag{I.36}$$

Maximizing the ELBO optimizes concurrently both the inference model and the generative model. The marginal likelihood $p_{\boldsymbol{\theta}}(\boldsymbol{\theta})$ is maximized, meaning the generated data maximally resembles the input data. Meanwhile, the distance between the approximated and the true posterior is minimized. Changing the prior distribution in the ELBO, $p_{\boldsymbol{\theta}}(\boldsymbol{z}) := p(\boldsymbol{z})$, the

overall loss function of a VAE is given by:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = -\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) + D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z})). \tag{I.37}$$

The first term is a reconstruction loss, which encourages the decoder to reconstruct the input data from the latent variables. The second term is a regularization term, which forces the approximate posterior to be close to the prior distribution.

VAEs have proven to be versatile and powerful in diverse applications, such as representation learning, anomaly detection, and generative modeling (Mienye and Swart, 2025). To enable VAEs to perform Bayesian inference of variables $\boldsymbol{x}$ given observations $\boldsymbol{y}$, the observations should be inserted into both the encoder and decoder as conditions. This method is named as conditional VAE (Sohn et al., 2015). The inference model becomes $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{y})$, the prior distribution becomes $p(\boldsymbol{z}|\boldsymbol{y})$, and the generative model becomes $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}|\boldsymbol{y})$. The conditions are often constrained to be low-dimensional variables, such as class labels (Sohn et al., 2015) or physical parameters (Martínez-Palomera et al., 2020). For high-dimensional and complex conditions, a powerful decoder is required to capture the relationship between the variables and observations. In this case, the latent variables may be ignored and thus the variability losses. This issue is known as posterior collapse (Bowman et al., 2016; McCarthy et al., 2020), which limits the application of VAEs in conditional inference tasks.

2.4 Normalizing Flow

Another deep probabilistic model is the Normalizing Flows (NFs). Rezende and Mohamed (2016) proposed NF as a method for approximating posterior distributions by transforming a simple base distribution through a sequence of invertible mappings into complex target distributions.

2.4.1 Basics

We simplify a NF as an invertible and differentiable function $f : \mathbb{R}^D \to \mathbb{R}^D$ (as known as a diffeomorphism) that maps a random variable $\boldsymbol{z}$ in latent space to another random variable $\boldsymbol{x}$ in data space:

$$\boldsymbol{x} = f(\boldsymbol{z}), \tag{I.38a}$$

$$\boldsymbol{z} = f^{-1}(\boldsymbol{x}). \tag{I.38b}$$

The variable $\boldsymbol{x}$ follows an underlying distribution of data $p_X(\boldsymbol{x})$. The variable $\boldsymbol{z}$ is drawn from a simple base distribution $p_Z(\boldsymbol{z})$, such as a standard normal distribution. Understanding the transformation between the two density functions is essential for comprehending NFs.

Change of Variables

According to the change of variables formula (Schwartz, 1954), when both the latent and data space are smooth and the function f is differentiable, we have:

$$\int \Psi(\boldsymbol{x})d\boldsymbol{x} = \int \Psi(\boldsymbol{x} = f(\boldsymbol{z})) \det |\boldsymbol{J}_f(\boldsymbol{z})| \, d\boldsymbol{z}, \tag{I.39}$$

where Ψ is a continuous function, $\boldsymbol{J}_f(\boldsymbol{z})$ is the Jacobian of the mapping f and $\det|\cdot|$ stands for determinant:

$$\det |\boldsymbol{J}_f(\boldsymbol{z})| = \det \left| \frac{\partial f(\boldsymbol{z})}{\partial \boldsymbol{z}} \right|. \tag{I.40}$$

This equation (I.39) implies probability conservation under transformation. In probability theory, the Radon–Nikodym derivative (Durrett, 1996) is a way to express how one probability measure changes with respect to another, which can be seen as a generalization of the relationship between Cumulative Distribution Function (CDF) and Probability Density Function (PDF). For a measurement set A for the variable $\boldsymbol{x}$, the distribution is:

$$P(X \in A) = \int_A p_X(\boldsymbol{x})d\boldsymbol{x}. \tag{I.41}$$

Applying the Equation I.38b, we have:

$$P(X \in A) = P(Z \in f^{-1}(A)), \tag{I.42a}$$

$$\int_A p_X(\boldsymbol{x})d\boldsymbol{x} = \int_{f^{-1}(A)} p_Z(\boldsymbol{z})d\boldsymbol{z}. \tag{I.42b}$$

Using change of variables formula (I.39), the last integral becomes:

$$\int_{f^{-1}(A)} p_Z(\boldsymbol{z})d\boldsymbol{z} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| d\boldsymbol{x}, \tag{I.43}$$

where the Jacobian determinant of the inverse mapping f^{-1} is:

$$\det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| = \det \left| \frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}} \right| = \det \left| \frac{\partial \boldsymbol{z}}{\partial f(\boldsymbol{z})} \right| = \frac{1}{\det |\boldsymbol{J}_f(\boldsymbol{z})|}. \tag{I.44}$$

Fusing the equations I.42b and I.43 together, we get the integral equality on the measurement set A:

$$\int_A p_X(\boldsymbol{x})d\boldsymbol{x} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| d\boldsymbol{x}. \tag{I.45}$$

Since the integrands are continuous almost everywhere, the derivatives also hold the equality:

$$p_X(\boldsymbol{x}) = p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| . \tag{I.46}$$

Substituting $\boldsymbol{x}$ and $\boldsymbol{z}$ with equations I.38a and I.38b respectively, it can also be formulated using the forward mapping f:

$$p_Z(\boldsymbol{z}) = p_X(f(\boldsymbol{z})) + \det \left| \boldsymbol{J}_f(\boldsymbol{z}) \right| . \tag{I.47}$$

These two equations are declared as the change of variables applied for NFs. They are often formulated in logarithm function:

$$\log p_X(\boldsymbol{x}) = \log p_Z(f^{-1}(\boldsymbol{x})) + \log \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| , \tag{I.48}$$

$$\log p_Z(\boldsymbol{z}) = \log p_X(f(\boldsymbol{z})) + \log \det \left| \boldsymbol{J}_f(\boldsymbol{z}) \right| . \tag{I.49}$$

From the above derivation of the formulas, we can infer three important properties of NFs:

(1) The transformation function f is differentiable.

(2) The transformation function f is invertible. Or from the definition in set theory, f is bijective, which can be interpreted as globally invertibility.

(3) The Jacobian determinant should be efficient to compute.

On top of them, another important property the transformation f should be fulfilled is composability (Papamakarios et al., 2021). Given a series of transformations $f_L \circ \cdots \circ f_1$, the following equations hold:

$$\boldsymbol{x} = f_L(\cdots f_1(\boldsymbol{z})), \tag{I.50a}$$

$$(f_L \circ \cdots \circ f_1)^{-1} = (f_1)^{-1} \circ \cdots \circ (f_L)^{-1}, \tag{I.50b}$$

$$\boldsymbol{J}_{f_L \circ \cdots \circ f_1}(\boldsymbol{z}) = \boldsymbol{J}_{f_L}(f_{L-1}(\boldsymbol{z})) \cdots \boldsymbol{J}_{f_1}(\boldsymbol{z}), \tag{I.50c}$$

where the total Jacobian determinant is the product of each intermediate ones.

Using NFs for modeling and inference

The invertibility also provides us easy way to fit NF models, i.e., minimizing the divergence between the true data distribution and the estimated one. For a NF with parameters $\boldsymbol{\theta}$, the KL-divergence is given by:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x}) || p_X(\boldsymbol{x}; \boldsymbol{\theta})) = \mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x}) - \mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x}; \boldsymbol{\theta}). \tag{I.51}$$

The first term can be considered constant while optimizing the parameters $\boldsymbol{\theta}$. The second term can be estimated by Monte Carlo methods:

$$\mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x};\boldsymbol{\theta}) \approx \frac{1}{M} \sum_{m=1}^{M} \log p_X(\boldsymbol{x}^{(m)};\boldsymbol{\theta}), \tag{I.52}$$

using a dataset $[\boldsymbol{x}^{(m)}]$, $m = 1, \ldots, M$. This is equivalent to the maximum likelihood estimation. Expanding $p_X(\boldsymbol{x};\boldsymbol{\theta})$ with Equation I.46, the KL-divergence becomes:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x})||p_X(\boldsymbol{x};\boldsymbol{\theta})) \approx const. - \frac{1}{M} \sum_{m=1}^{M} [\log p_Z(f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{x}^{(m)})) + \log\det \left| \boldsymbol{J}_{f_{\boldsymbol{\theta}}^{-1}}(\boldsymbol{x}^{(m)}) \right|], \tag{I.53}$$

where $p_Z(\boldsymbol{z})$ is a standard normal distribution. Then, the parameters are optimized by stochastic gradient-based methods.

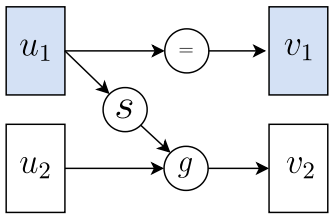
The loss function of NFs suggests that during training, the learned latent representation is compared with the true latent distribution, while the term of Jacobian determinant ensures this comparison to be transformed back to the data space. Recalling the VAE, the first term can be seen as inference, while the second term can be implicitly regarded as "preparation for generation". After the NF model is well-trained, new data can be generated by sampling the latent variables and transforming them via the forward mapping. It is important to emphasize that, NFs are conventionally trained in one direction at a time. Simultaneously bidirectional training would necessitate computing gradients through the composition of a function and its inverse, leading to problematic gradient flows of the form $\partial L / \partial \boldsymbol{\theta}_f(\boldsymbol{\theta}_{\boldsymbol{f}^{-1}})$, which creates numerical instabilities and computational inefficiencies.

A variety of architectures have been developed to realize NFs. A GitHub website (Riebesell et al., 2023) summarizes the key publications and implementations in this domain. As outlined in the comprehensive review by Kobyzev et al. (2021), successful architectures include coupling flows, autoregressive flows, residual flows, and infinitesimal (continuous) flows. Throughout this work, we focus primarily on coupling flows, which offer structural invertibility and have been extensively utilized in NFs. Infinitesimal flows characterize distributional transformations through continuous dynamics, establishing deep theoretical connections with other probabilistic frameworks such as diffusion models. Thus, we also introduce continuous NFs following the description of coupling flows.

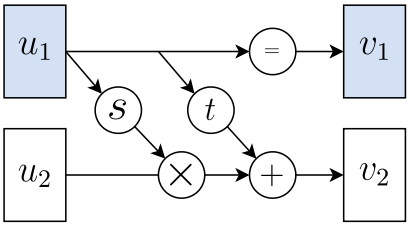
2.4.2 Coupling Flows

The basic idea of the coupling flow is partitioning the input variables into two subsets, one subset of which is processed by neural networks and also concurrently directly passed through to the output variables, while the other subset is combined with the processed

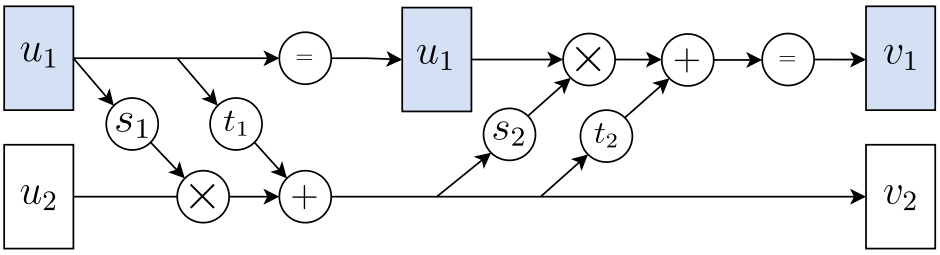
subset through affine functions.



(a) The architecture of a Non-linear Independent Component Estimation (NICE) layer. s is a neural network, and g is an affine function.



(b) The architecture of a Real-Valued Non-Volume Preserving (RealNVP) layer. s and t are neural networks. The symbol × stands for element-wise product by the exponential function over input.



(c) The architecture of an Invertible Neural Network (INN) layer. s_1, s_2, t_1, and t_2 are neural networks. The symbol × stands for element-wise product by the exponential function over input.

Figure I.3: The architectures of coupling flows. The input of the layer is $\boldsymbol{u}$ and the output is $\boldsymbol{v}$.

The first proposed coupling flow is NICE (Dinh et al., 2015). The input variables $\boldsymbol{u}$ are split into two parts $[\boldsymbol{u}_1, \boldsymbol{u}_2]$, and the corresponding output variables are $\boldsymbol{v} = [\boldsymbol{v}_1, \boldsymbol{v}_2]$. The architecture in the forward direction is formulated by the following functions:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = g(\boldsymbol{u}_2; s(\boldsymbol{u}_1)), \tag{I.54}$$

where s is a neural network and g is an affine function $g(a; b) = a + b$ or $a \odot b$. The forward architecture is illustrated in Figure I.3a. Then, the inverse functions are easily calculated:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = g^{-1}(\boldsymbol{v}_2; s(\boldsymbol{u}_1)). \tag{I.55}$$

From the above functions, the Jacobian matrix is:

$$\frac{\partial \boldsymbol{v}}{\partial \boldsymbol{u}} = \begin{bmatrix} \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_2 \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_2 \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2 \end{bmatrix} \tag{I.56}$$

Because of the triangle matrix, the determinant is $\partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2$. By composing multiple coupling layers, we can construct increasingly complex hierarchical transformations. Since each coupling layer preserves a subset of its input unchanged, it is essential to alternate the roles of the partitioned subsets across successive layers. This ensures that all input dimensions are eventually processed. Analyzing the Jacobian reveals that a minimum of three coupling layers is required for full inter-dimensional interaction. In practice, we typically employ four layers to achieve sufficient expressiveness.

An improved architecture is RealNVP, where one part of the variables is processed by two neural networks. The forward transformation is:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = \boldsymbol{u}_2 \odot \exp(s(\boldsymbol{u}_1)) + t(\boldsymbol{u}_1). \tag{I.57}$$

And the inverse transformation is:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = (\boldsymbol{v}_2 - t(\boldsymbol{u}_1)) \odot \exp(-s(\boldsymbol{u}_1)). \tag{I.58}$$

The Jacobian determinant $\det |\partial \boldsymbol{v}/\boldsymbol{u}|$ is then $\exp(\Sigma_i s(\boldsymbol{u}_{1,i}))$. The architecture is shown in Figure I.3b. To interact all variables in one block, INN (Ardizzone et al., 2019a) extends RealNVP by another same coupling block, as shown in Figure I.3c.

A further enhancement of RealNVP is Glow (Kingma and Dhariwal, 2018), which inserts an invertible normalization layer and an invertible permutation layer before affine transformation. Hierarchical Invertible Neural Transport (HINT) (Kruse et al., 2021) partitions the inputs of coupling blocks recursively to construct a dense, triangular Jacobian. Another method to increase interaction among variables is Masked Autoregressive Flow (MAF) (Papamakarios et al., 2018), which can be considered as a generalized RealNVP that leverages autoregressive transformations between variables.

Such coupling flows and autoregressive flows build step-wise invertible transformations between distributions, enabling tractable Jacobian calculation and density estimation. A notable drawback lies in the strict architectural constraint on coupling blocks. To release this limitation, residual flows (Behrmann et al., 2019; Chen et al., 2020) introduce invertible mapping through specified residual blocks. A residual block (He et al., 2015) takes the form $f(x) = x + F(x)$, where F is a neural network. When F is constrained to be Lipschitz continuous and the Lipschitz constant is smaller than 1 (explained in 2.4.3), the function $f(x)$ is invertible. The residual block can be seen as an Euler discretization

of an ordinary differential equation (ODE) (Lu et al., 2020; Ruthotto and Haber, 2018; Haber and Ruthotto, 2017), which is the foundation of continuous NFs.

2.4.3 Continuous Flows

When the transformation from latent space to data space is modeled by a continuous dynamical system, the NF is called a continuous NF. The continuous dynamical system can be formulated by a Neural ODE, which is introduced as follows.

Neural ODEs

For a dynamical system with states $\boldsymbol{h}$, an ODE is:

$$\frac{d\boldsymbol{h}(t)}{dt} = f(\boldsymbol{h}(t), t; \boldsymbol{\theta}), \tag{I.59}$$

where $\boldsymbol{\theta}$ denotes system parameters. When the ODE is modeled by neural networks, it becomes a Neural ODE (Chen et al., 2019). Given the initial state $\boldsymbol{h}(0)$, the final state $\boldsymbol{h}(T)$ is the solution of the initial value problem at the time step T using a numerical solver. The loss function of training a Neural ODE is:

$$L(\boldsymbol{h}(T)) = L(\int_0^T f(\boldsymbol{h}(t), t; \boldsymbol{\theta})dt), \tag{I.60}$$

where f is a neural network parameterized by $\boldsymbol{\theta}$. A straightforward application of Neural ODEs is learning continuous time-series models. Moreover, when the time step is interpreted as architectural depth, the distributional transformation in NFs can also be constructed with a Neural ODE. The Neural ODE replaces an infinite number of transformations by change of variables with a single neural network, which represents the derivative of transformation at each depth. This enables more efficient and expressive probabilistic modeling and inference.

A central technical challenge in training continuous-depth networks lies in executing reverse-mode differentiation (commonly known as backpropagation) through the ODE solver. This process requires computing gradients with respect to network parameters at various time points backwards from $t = T$ to $t = 0$, resulting in large memory cost to store the intermediate states. The solution is using the adjoint method (Bittner, 1963; Gavrilović et al., 1963). The principle of the adjoint method is to solve a backward ODE alongside the forward ODE whenever intermediate states are required. Compared to storing all intermediate states of the forward ODE, the adjoint method reduces memory cost at the expense of increased computational cost. This trade-off makes it more suitable for long

sequences and high-dimensional variables, but less efficient for computationally complex network architectures.

Lipschitz continuity

We define a continuous NF as a Neural ODE:

$$\frac{d\boldsymbol{z}(t)}{dt} = f(\boldsymbol{z}, t; \boldsymbol{\theta}), \tag{I.61}$$

where $\boldsymbol{z}(t)$ follows the density $p(\boldsymbol{z}(t))$ at an instantaneous step $t \in [t_0, t_1]$. The NF f is parameterized by $\boldsymbol{\theta}$. To apply the Neural ODE as continuous NFs, the function f should be uniformly Lipschitz continuous in the states $\boldsymbol{z}(t)$ and continuous in depth t.

The definition of a Lipschitz continuous function is referred to the work by Khromov and Singh (2024).

Definition 1 ***(Lipschitz continuous function)*** *A function $f : \mathbb{R}^d \to \mathbb{R}^K$, defined on some domain $dom(f) \subseteq \mathbb{R}^d$, is called C-Lipschitz continuous w.r.t. some α-norm, if $\forall \boldsymbol{x}, \boldsymbol{y} \in dom(f) : \|f(\boldsymbol{x}) - f(\boldsymbol{y})\|_\alpha \leq C \|\boldsymbol{x} - \boldsymbol{y}\|_\alpha$. C is called Lipschitz constant of f and $0 < C < \inf$.*

The Lipschitz condition guarantees that there exists a unique solution curve $\boldsymbol{t}$ for every initial condition $\boldsymbol{z}(0)$. This is known as the Picard–Lindelöf theorem. Assuming the dynamics function f is smooth, then it is a diffeomorphism, which means we also get the same $\boldsymbol{z}(0)$ by running the ODE backwards. Recalling the residual flows, where the Lipschitz constant must be smaller than 1, invertibility is ensured naturally by a smooth and Lipschitz continuous ODE. In practice, neural networks with smooth continuous activation functions satisfy this requirement.

Continuous NFs

Besides for the ODE I.61, the log-density of states changes according to another ODE, the instantaneous change-of-variables formula (Chen et al., 2019):

$$\frac{\partial \log p(\boldsymbol{z}(t))}{\partial t} = -\mathrm{Tr}\left(\frac{\partial f(\boldsymbol{z}(t), t; \boldsymbol{\theta})}{\partial \boldsymbol{z}(t)}\right). \tag{I.62}$$

The trace can be approximated through Skilling-Hutchinson trace estimator (Skilling, 1989; Hutchinson, 1990; Adams et al., 2018):

$$\mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) = \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[\boldsymbol{\epsilon}^T \frac{\partial f}{\partial \boldsymbol{z}(t)} \boldsymbol{\epsilon}\right], \tag{I.63}$$

where $\boldsymbol{\epsilon}$ is a noise vector sampled from a standard normal distribution and kept unchanged to maintain determinism of the ODE. The part of vector-Jacobian product $\boldsymbol{\epsilon}^T \partial f / \partial \boldsymbol{z}(t)$ is computed with automatic differentiation.

For a continuous NF between the data distribution $p(\boldsymbol{x}) = p(\boldsymbol{z}(t_1))$ and the latent distribution $p_{Z_0}(\boldsymbol{z}_0) = p(\boldsymbol{z}(t_0)) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, we define the augmented states as $[\boldsymbol{z}(t), \log p(\boldsymbol{x}) - \log p(\boldsymbol{z}(t))]$. Then, building the continuous NF is equivalent to solving the initial value problem (Grathwohl et al., 2018):

$$\underbrace{\begin{bmatrix} \boldsymbol{z}_0 \\ \log p(\boldsymbol{x}) - \log p_{Z_0}(\boldsymbol{z}_0) \end{bmatrix}}_{\text{solutions}} = \int_{t_1}^{t_0} \underbrace{\begin{bmatrix} f(\boldsymbol{z}(t), t; \theta) \\ -\mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) \end{bmatrix}}_{\text{dynamics}} dt, \underbrace{\begin{bmatrix} \boldsymbol{z}(t_1) \\ \log p(\boldsymbol{x}) - \log p(\boldsymbol{z}(t_1)) \end{bmatrix}}_{\text{initial values}} = \begin{bmatrix} \boldsymbol{x} \\ 0 \end{bmatrix}. \quad \text{(I.64)}$$

It is worth noting that the integral is backwards calculated from t_1 to t_0, meaning from the data side to the latent side. The time span can also be optimized to further improve the computational efficiency (Du et al., 2022).

Continuous NFs release the architectural constraints of discrete flows, allowing more flexible and expressive density estimation through the integration of Neural ODEs. This improved expressiveness, however, comes at a cost: computing the Jacobian trace and the associated ODE integrals is significantly more expensive than in stacked coupling flows.

2.4.4 Flow Matching

Flow Matching (FM) (Lipman et al., 2023) is a novel approach for training continuous NFs. The key idea is to learn a vector field that directly matches the step-by-step transport (infinitesimal transport) between the data distribution and the latent distribution. This is achieved by minimizing the expected squared error between the learned vector field and the true velocity field defined by the optimal transport plan. The optimal transport plan is normally a linear interpolation between the latent samples $\boldsymbol{z}(t_0)$ and the data samples $\boldsymbol{x} = \boldsymbol{z}(t_1)$:

$$\boldsymbol{v}^{OT}(\boldsymbol{z}, t) = \frac{\boldsymbol{z}(t_1) - \boldsymbol{z}(t_0)}{t_1 - t_0}. \quad \text{(I.65)}$$

The training objective is formulated as:

$$L(\boldsymbol{\theta}) = \mathbb{E}_{t \sim \mathcal{U}(0,1), \boldsymbol{z} \sim p_t(\boldsymbol{z})} \left[\left\| v_{\boldsymbol{\theta}}(\boldsymbol{z}, t; \boldsymbol{\theta}) - \boldsymbol{v}^*(\boldsymbol{z}, t) \right\|^2 \right], \quad \text{(I.66)}$$

where $v_{\boldsymbol{\theta}}(\boldsymbol{z}, t; \boldsymbol{\theta})$ is the learned vector field parameterized by $\boldsymbol{\theta}$, and $\boldsymbol{v}^*(\boldsymbol{z}, t)$ is the true velocity field derived from the optimal transport plan. Simply put, the FM loss regresses the vector field $\boldsymbol{v}^*$ with a neural network $v_{\boldsymbol{\theta}}$. Upon reaching zero loss, the learned continuous NF model will generate data samples by integrating the learned vector field from t_0 to t_1.

The main difference from other NFs is that FM is not trained by maximizing likelihood via change of variables between distributions. Instead, it focuses on learning the vector field that defines the continuous transformation between distributions. This approach avoids the computational overhead of solving ODEs during training, making it more efficient and scalable to high-dimensional data. On the other hand, FM cannot be directly applied to exact density estimation because it does not transform the data distribution into a known latent distribution. The sample density it generates is only an approximation of the data density. For exact estimation, FM must invert the generative process via a backward ODE, which makes it computationally heavier than the single-pass inference of other NFs.

2.4.5 Conditional Normalizing Flows

Such capabilities of estimating complex densities are desirable in Bayesian inference of the posterior distribution of variables given observations $p(\boldsymbol{x}|\boldsymbol{y})$. Similar to VAE, amortized Bayesian inference (Radev, 2021) is applied in NF to achieve a general posterior estimation across all data points. Obtaining the posterior distribution is represented by a conditional density transformation via a Conditional Normalizing Flow (CNF). The change of variables for a CNF extends the standard one (I.46) by the condition $\boldsymbol{y}$:

$$p_{X|Y}(\boldsymbol{x}|\boldsymbol{y}) = p_{Z|Y}(f^{-1}(\boldsymbol{x},\boldsymbol{y})|\boldsymbol{y}) \det \left| \frac{\partial f^{-1}(\boldsymbol{x},\boldsymbol{y})}{\partial \boldsymbol{x}} \right|, \tag{I.67}$$

where the mapping $f : Z \to X \times Y$ is bijective between X and Z, and $\boldsymbol{z} = f^{-1}(\boldsymbol{x},\boldsymbol{y})$.

Designing such a CNF requires addressing two key considerations in advance: (1) how to manage the conditioned latent variables, and (2) how to incorporate conditions into the flow architecture while preserving invertibility. Winkler et al. (2023) proposed to build the conditional prior $p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y})$ as a diagonal Gaussian:

$$p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y}) = \mathcal{N}\boldsymbol{z}|\mu(\boldsymbol{y}, \mathrm{diag}(\sigma^2(\boldsymbol{y}))). \tag{I.68}$$

For simpler conditioning scenarios, incorporating the conditional prior could accelerate the transformation through leveraging the information from conditions. For complex conditions, summarizing the conditions through mean and variance may result in information loss, thereby degrading posterior estimation quality. This limitation likely explains why most existing work on CNFs adopts a standard Gaussian as the latent base distribution, bypassing the need for an explicitly conditioned prior and relying instead on the normalizing flows to capture conditional information.

In terms of architectures of CNFs, a direct adaptation based on the INN is the conditional Invertible Neural Network (cINN) (Ardizzone et al., 2019b). It takes the

condition as augmented input to the neural networks. In cases of higher-dimensional conditions, such as images, Anantha Padmanabha and Zabaras (2021) apply a staged cINN coupled with a staged condition network, wherein the conditions are progressively processed, and each intermediate representation is fed into its corresponding cINN block.

In scenarios involving complex conditions and relatively simple target variables, BayesFlow (Radev et al., 2020) offers a general framework by extracting informative representations through a summary network and incorporating them, alongside the inputs, as augmented inputs to the cINN. This framework is particularly well-suited for parameter identification from time series observations, where the summary network is typically chosen as a time series modeling architecture, such as RNN. The architecture is illustrated in Figure I.4. Both the summary network and cINN are optimized together by minimizing the KL-divergence between the true posterior density and the estimated one (similar to Equation I.53):

$$D_{\mathrm{KL}}(p_{X|Y}(\boldsymbol{x}|\boldsymbol{y})||p_{X|Y}(\boldsymbol{x}|\boldsymbol{y};\boldsymbol{\theta},\boldsymbol{\phi})) \approx const.- \\ \frac{1}{M}\sum_{m=1}^{M}[\log p_Z(f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))) + \log\det\left|\boldsymbol{J}_{f_{\boldsymbol{\theta}}^{-1}}(\boldsymbol{x}^{(m)}, h_{\boldsymbol{\phi}}(\boldsymbol{x}^{(m)}))\right|], \quad \text{(I.69)}$$

where f is the cINN parameterized by $\boldsymbol{\theta}$ and h is the summary network parameterized by $\boldsymbol{\phi}$.

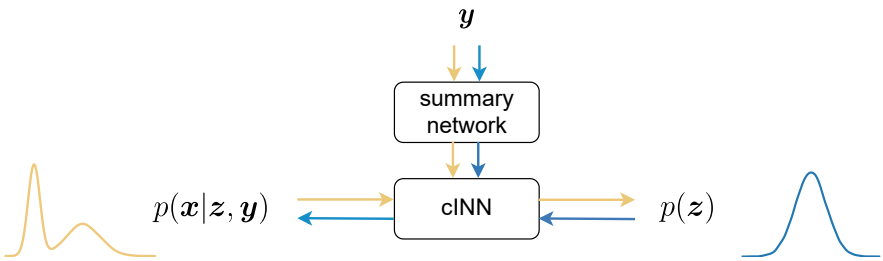


Figure I.4: The architecture of BayesFlow. The blue arrows indicate the forward direction for training BayesFlow, while the yellow arrows indicate the reverse direction for inference. The summary network extracts informative representations from the observations, which are then fed into the cINN alongside the target variables.

The joint optimization approach encourages the summary network to deliver maximally useful information to the cINN thus improve the inference capability. In this work, we adopt the BayesFlow framework as a flexible and scalable solution for amortized Bayesian inference, adapting its summary network to accommodate diverse time series observations. This framework forms the foundation of our proposed methods for parameter identification,

system dynamics modeling, and state estimation.

2.5 From Bayesian Inference to Control

This section bridges the Bayesian inference methodologies discussed above to their practical applications in control systems. We first review how Bayesian approaches are employed for system identification and state estimation. Next, we introduce Model Predictive Control (MPC), which is a widely used model-based control technique for optimal decision-making based on the system identification results. Finally, we discuss Reinforcement Learning (RL) as a data-driven control paradigm, highlighting its ability to learn control policies directly from interactions with the environment without requiring explicit system models. Together, these approaches illustrate the synergy between Bayesian inference and modern control strategies, enabling robust and adaptive solutions for complex dynamical systems.

2.5.1 Bayesian System Identification

An essential goal of system modeling and identification is to provide a rigorous framework for predicting and controlling the future evolution of system outputs. For any control strategy, the decision-maker needs to predict how the system will behave under any possible control input. This requires access to the joint probability distribution of the future input-output sequence, which can be factorized using the chain rule of probability (Peterka, 1981):

$$p(\mathcal{D}_{(t_1:t_N)}|\mathcal{D}_{t_0}) = \prod_{n=1}^{N} p(\boldsymbol{y}_{t_n}|\boldsymbol{u}_{t_n}, \mathcal{D}_{t_{n-1}}) p(\boldsymbol{u}_{t_n}|\mathcal{D}_{t_{n-1}}), \tag{I.70}$$

where $\mathcal{D}_{t_n}$ denotes the input-output pair at the time step t_n, i.e., $\mathcal{D}_{t_n} = \{\boldsymbol{u}_{t_n}, \boldsymbol{y}_{t_n}\}$. The conditional probability density $p(\boldsymbol{u}_{t_n}|\mathcal{D}_{t_{n-1}})$ describes the process of determining the input $\boldsymbol{u}_{t_n}$ on the basis of the known past history. The left part $p(\boldsymbol{y}_{t_n}|\boldsymbol{u}_{t_n}, \mathcal{D}_{t_{n-1}})$ represents the dependence of the output on the corresponding input and the known past history, which can be interpreted as a system model or function to predict output with given control input. Identifying this system consists of two steps: model selection and parameter identification. The process of finding the most plausible model from a group of candidates is model selection. Then, system identification focuses on inferring parameters of a chosen system model.

Bayesian model selection

Among different model structures $\{\mathcal{M}_i\}$, $i = 1, \ldots, |\mathcal{M}|$, the probability density of the model $\mathcal{M}_i$ given the data $\mathcal{D}$ measures whether the model is suitable for the dataset, which

can also be written using Bayes' theorem:

$$p(\mathcal{M}_i|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D})}. \tag{I.71}$$

Model selection is also as known as model comparison. The relative probability density of two different models given same data $\mathcal{D}$ is:

$$\frac{p(\mathcal{M}_i|\mathcal{D})}{p(\mathcal{M}_j|\mathcal{D})} = \frac{p(\mathcal{D}|\mathcal{M}_i)p(\mathcal{M}_i)}{p(\mathcal{D}|\mathcal{M}_j)p(\mathcal{M}_j)}. \tag{I.72}$$

Since a model has multiple undetermined parameters, the distribution $p(\mathcal{D}|\mathcal{M})$ is obtained by integral of all variants of parameters. The integral is usually untractable. Instead, Deviance Information Criterion (DIC) (Spiegelhalter et al., 2002, 2014; Li et al., 2025) is applied as the model selection criterion. A deviance is defined as follows:

$$\text{dev}(\boldsymbol{\theta}) = -2\log p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M}), \tag{I.73}$$

where $\boldsymbol{\theta}$ denotes the parameters of the model $\mathcal{M}$. Then, the DIC is defined as:

$$\text{DIC}(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\text{dev}(\boldsymbol{\theta})] + P_D(\mathcal{M}), \tag{I.74}$$

where the first term is interpreted as a Bayesian measure of model fit:

$$\mathbb{E}_{\mathcal{M}}[\text{dev}(\boldsymbol{\theta})] = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\text{dev}(\boldsymbol{\theta})d\boldsymbol{\theta}, \tag{I.75}$$

and the second term is used to measure the model complexity:

$$P_D(\mathcal{M}) = \mathbb{E}_{\mathcal{M}}[\text{dev}(\boldsymbol{\theta})] - \text{dev}(\overline{\boldsymbol{\theta}}), \quad \overline{\boldsymbol{\theta}} = \int p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})\boldsymbol{\theta}d\boldsymbol{\theta}. \tag{I.76}$$

The posterior $p(\boldsymbol{\theta}|\mathcal{D}, \mathcal{M})$ is obtained by MCMC. DIC balances the goodness of fit (how well the model explains the data) and the model complexity (the number of parameters in the model). A lower DIC value indicates better model fit for the given dataset.

Bayesian Parameter Identification

We first define a chosen system model with unknown parameters $\boldsymbol{\theta}$:

$$\boldsymbol{y}_t = f(\boldsymbol{u}_t, \boldsymbol{\theta}) + \boldsymbol{\epsilon}_t. \tag{I.77}$$

The optimal parameters are selected by maximum a posteriori (MAP) estimation:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|\mathcal{D}). \tag{I.78}$$

Applying Bayes' rule (I.2), it becomes:

$$\boldsymbol{\theta}^*_{MAP} = \arg\max_{\boldsymbol{\theta}} [\prod_{\{\boldsymbol{u}_i,\boldsymbol{y}_i\}\in\mathcal{D}} p(\boldsymbol{y}_i|\boldsymbol{u}_i,\boldsymbol{\theta})]p(\boldsymbol{\theta}), \tag{I.79}$$

where $p(\mathcal{D})$ is dropped off since it is a constant with respect to $\boldsymbol{\theta}$. The noise $\boldsymbol{\epsilon}_t$ is commonly assumed to be Gaussian. Thus, the distribution $p(\boldsymbol{y}_i|\boldsymbol{u}_i,\boldsymbol{\theta})$ has a closed form, which makes this term equivalent to a weighted mean squared error. The term of the prior density $p(\boldsymbol{\theta})$ performs as $L2$ regularization. On top of the estimate of parameters, the confidence interval of parameters can also be obtained from the posterior distribution.

2.5.2 Bayesian State Estimation

Applying the estimated posterior of system parameters again as a prior, and updating the posterior with new data, this is the basic idea of Bayesian state estimation (Barker et al., 1995). State estimation involves inference of the internal system states, which are not directly measurable. Expressing the state estimation problem in a probabilistic framework, the goal is to infer the posterior distribution of the states given observations $p(\boldsymbol{x}_{0:t}|\boldsymbol{y}_{1:t})$. Assuming a Markovian system, the posterior of the state chain is factorized as:

$$\begin{aligned} p(\boldsymbol{x}_{0:t}|\boldsymbol{y}_{1:t}) = p(\boldsymbol{x}_{0:t}|\boldsymbol{y}_t,\boldsymbol{y}_{1:t-1}) &= \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_{0:t},\boldsymbol{y}_{1:t-1})p(\boldsymbol{x}_{0:t}|\boldsymbol{y}_{1:t-1})}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} \quad \text{[Bayes' rule]} \\ &= \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_{0:t}|\boldsymbol{y}_{1:t-1})}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} \quad \text{[Markov property]} \\ &= \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} p(\boldsymbol{x}_{0:t-1}|\boldsymbol{y}_{1:t-1}), \end{aligned} \tag{I.80}$$

where $p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ is referred as evolution model, or state transition model, and $p(\boldsymbol{y}_t|\boldsymbol{x}_t)$ is called observation model, or measurement model. The posterior $p(\boldsymbol{x}_{0:t-1}|\boldsymbol{y}_{1:t-1})$ is the result of the previous time step. The equation above describes a recursive process of state estimation.

Bayesian Filters

Considering only one state $\boldsymbol{x}_t$ at a time, the problem is called marginal state estimation, or filtering. The equation becomes:

$$\begin{aligned} p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t}) &= \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1})}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} \\ &= \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_t)}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} \int p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})p(\boldsymbol{x}_{t-1}|\boldsymbol{y}_{1:t-1})d\boldsymbol{x}_{t-1}. \end{aligned} \tag{I.81}$$

Usually, calculation of the above equation is divided into two steps: prediction and update. In the prediction step, the result of state transition from $\boldsymbol{x}_{t-1}$ to $\boldsymbol{x}_t$ is computed through the evolution model. In the update step, the last observation likelihood $p(\boldsymbol{y}_t|\boldsymbol{x}_t)$ weights the predicted belief. These two steps build the Bayesian filtering methods: Kalman filters and particle filters (Chen, 2003).

The Kalman Filter (KF) (Kalman, 1960) can be directly derived from Equation I.81 under the assumptions of linear systems and Gaussian noise. For a linear system:

$$\boldsymbol{x}_t = F_t\boldsymbol{x}_{t-1} + \boldsymbol{w}_t, \quad \boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{w_t}), \tag{I.82a}$$

$$\boldsymbol{y}_t = H_t\boldsymbol{x}_t + \boldsymbol{v}_t, \quad \boldsymbol{v}_t \sim \mathcal{N}(\boldsymbol{0}, \Sigma_{v_t}), \tag{I.82b}$$

the iterative algorithm of filtering is:

- Prediction:

$$\boldsymbol{x}'_t = F_t\boldsymbol{x}_{t-1}, \tag{I.83a}$$

$$\Sigma'_t = F_t\Sigma_{t-1}F_t^T + \Sigma_{w_t}. \tag{I.83b}$$

- Update:

$$K_t = \Sigma'_t H_t^T(H_t\Sigma'_t H_t^T + \Sigma_{v_t})^{-1}, \tag{I.84a}$$

$$\boldsymbol{x}_t = \boldsymbol{x}'_t + K_t(\boldsymbol{y}_t - H_t\boldsymbol{x}'_t), \tag{I.84b}$$

$$\Sigma_t = (I - K_tH_t)\Sigma'_t. \tag{I.84c}$$

For nonlinear systems, the Extended Kalman Filter (EKF) (Simon, 2006) obtains the matrices F_t and H_t by linearizing the nonlinear functions at the current estimate, i.e., $F_t = \nabla_{\boldsymbol{x}} f_t|_{\boldsymbol{x}_t}$ and $H_t = \nabla_{\boldsymbol{x}} h_t|_{\boldsymbol{x}_t}$. In such a way, the EKF can be considered as the first-order approximation to the optimal terms. This approximation, however, can introduce large errors, which may lead to suboptimal performance and sometimes divergence of the

filter. The Unscented Kalman Filter (UKF) (Wan and Van Der Merwe, 2000) addressed the approximation issues by carefully choosing a minimal set of samples points, which are called sigma points. The sigma points completely capture the mean and covariance of states, which are viewed as Gaussian random variables. The sample points are then propagated through the nonlinear functions, and the mean and covariance of the estimate are reconstructed from the transformed points.

Relaxing the assumptions on model linearity and Gaussian noise, Particle Filter (PF) (Gordon et al., 1993; Elfring et al., 2021) is a more general Bayesian filtering method. The integral in Equation I.81 is approximated through Monte Carlo estimation, yielding an average of all weighted particles. Each particle represents a possible state of the system, and its weight reflects the likelihood of that state given the observations. In the prediction step, each particle is propagated through the system dynamics. In the update step, weights are adjusted according to each particle based on the likelihood of the observation given the particle's state. Unlike Kalman filters, in each iteration of the PF, a resampling step is included to avoid weight degeneracy, meaning that most particles carry very small weights over time. In the resampling step, particles are replaced with equally weighted ones, replicating high-weight particles and discarding low-weight ones.

In summary, KF is efficient and provides optimal state estimates for linear systems with Gaussian noise. EKF extends KF to nonlinear systems through linearization, but may suffer from approximation errors. UKF, instead of linearization, applies sigma points to better capture the mean and covariance of the state distribution, leading to more accurate estimates for highly nonlinear systems. PF is the most flexible, capable of handling arbitrary nonlinearities and non-Gaussian noise, but computationally expensive due to the need for numerous particles.

State-Space Models

For modeling system dynamics completely from data, State-Space Models (SSMs) (Durbin and Koopman, 2012) are widely used. An SSM consists of a latent state vector $\boldsymbol{x}_t$ and an observation vector $\boldsymbol{y}_t$. The state vector captures the internal dynamics of the system, while the observation vector represents the measurable outputs. $\boldsymbol{x}_t$ follows a Markov process, meaning that the previous state $\boldsymbol{x}_{t-1}$ carries all information to obtain the current state $\boldsymbol{x}_t$. $\boldsymbol{y}_t$ is solely dependent on the latent state $\boldsymbol{x}_t$. These assumptions lead to the following factorization of the joint distribution:

$$p(\boldsymbol{y}_{1:T}, \boldsymbol{x}_{0:T}) = p(\boldsymbol{x}_0) \prod_{t=1}^{T} p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) p(\boldsymbol{y}_t|\boldsymbol{x}_t). \tag{I.85}$$

In the simplest case, the state transition distribution $p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$ and the observation distribution $p(\boldsymbol{y}_t|\boldsymbol{x}_t)$ are assumed to be linear Gaussian models:

$$\boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, \Sigma_0), \tag{I.86a}$$

$$\boldsymbol{x}_t = \boldsymbol{F}\boldsymbol{x}_{t-1} + \boldsymbol{w}_t, \quad \boldsymbol{w}_t \sim \mathcal{N}(\mathbf{0}, \Sigma_{w_t}) \tag{I.86b}$$

$$\boldsymbol{y}_t = \boldsymbol{H}\boldsymbol{x}_t + \boldsymbol{v}_t, \quad \boldsymbol{v}_t \sim \mathcal{N}(\mathbf{0}, \Sigma_{v_t}). \tag{I.86c}$$

In the previous section, we have shown estimation of states using Kalman filters with known system matrices ($\boldsymbol{F}$, $\boldsymbol{H}$) and noise covariance matrices ($\Sigma_{w_t}, \Sigma_{v_t}$). When these matrices are unknown, they can be estimated from data using the Expectation-Maximization (EM) algorithm (Dempster et al., 1977). The EM algorithm iteratively maximizes the expected log-likelihood of the complete data (both observed and latent variables) given the current parameter estimates. The algorithm consists of two main steps:

- E-step: Compute the expectations of the latent states using the current parameter estimates:

  $$E_t = E[\boldsymbol{x}_t], \quad E_{t,t} = E[\boldsymbol{x}_t\boldsymbol{x}_t^T], \quad E_{t,t-1} = E[\boldsymbol{x}_t\boldsymbol{x}_{t-1}^T].$$

  The states come from running the Kalman filter (Equation I.83, I.84) on the current parameters.

- M-step: Update parameters using the expectations:

$$\boldsymbol{F} = \left(\sum_{t=1}^{T} E_{t,t-1}\right)\left(\sum_{t=1}^{T} E_{t-1,t-1}\right)^{-1}, \quad \boldsymbol{H} = \left(\sum_{t=0}^{T} \boldsymbol{y}_t E_t^T\right)\left(\sum_{t=1}^{T} E_{t,t}\right)^{-1},$$

$$\Sigma_{w_t} = \frac{1}{T}\left(\sum_{t=1}^{T} E_{t,t} - \boldsymbol{F}\sum_{t=1}^{T} E_{t,t-1}^T\right), \quad \Sigma_{v_t} = \frac{1}{T}\left(\sum_{t=0}^{T} \boldsymbol{y}_t\boldsymbol{y}_t^T - \boldsymbol{H}\sum_{t=0}^{T} \boldsymbol{y}_t E_t^T\right).$$

The E-step and M-step are iterated until convergence. The EM algorithm guarantees that the likelihood of the observed data does not decrease with each iteration, leading to convergence to a local maximum. Similar to Kalman filters, for nonlinear models, EKFs and UKFs are applied in the E-step; while for nonlinear and non-Gaussian SSMs, PFs are used to approximate the expectations in the E-step (Durbin and Koopman, 2000).

Stochastic Differential Equations

The core strength of SSMs lies in separating the latent states and noise, enabling capturing dynamics from the noisy observations. When we upgrade SSMs from discrete time to continuous time, Stochastic Differential Equations (SDEs) (Øksendal, 1985) are a

natural choice for modeling the evolution of latent states. A general form of an SDE is:

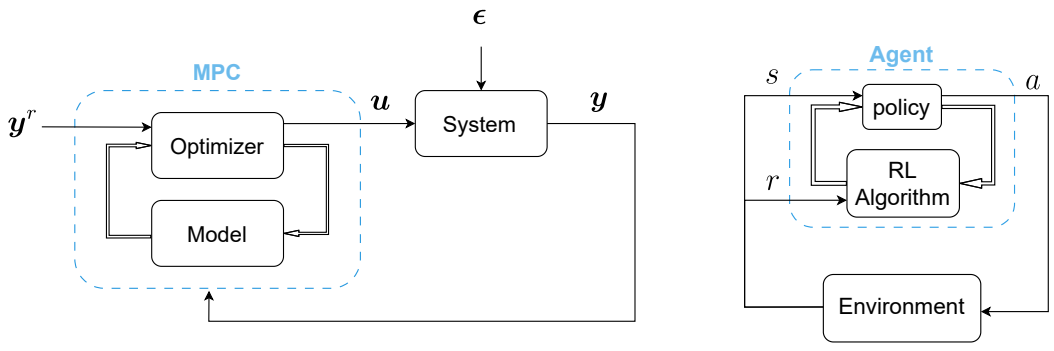
$$dX_t = \mu(X_t, t)dt + \sigma(X_t, t)dW_t, \tag{I.88}$$

where dX_t is the infinitesimal change in the state, $\mu(X_t, t)$ is the drift function representing the deterministic part of the dynamics, $\sigma(X_t, t)$ is the diffusion function representing the stochastic part, and W_t is a Wiener process (or Brownian motion) capturing the random fluctuations. A Wiener process is a standard stochastic process with independent, normally distributed increments and continuous paths. It has the following properties (Gabbiani and Cox, 2010):

- $W_0 = 0$.
- For any $0 \leq s < t$, the increment $W_t - W_s$ is normally distributed with mean 0 and variance $t - s$, i.e., $W_t - W_s \sim \mathcal{N}(0, (t-s)I)$.
- The process has independent increments, meaning that for any $0 \leq t_0 < t_1 < \ldots < t_n$, the increments $W_{t_1} - W_{t_0}, W_{t_2} - W_{t_1}, \ldots, W_{t_n} - W_{t_{n-1}}$ are independent random variables.

A stochastic process is a collection of random variables indexed by time. It can be seen as a continuous-time generalization of the state vector in the SSMs.

2.5.3 Model Predictive Control



(a) Schematic of MPC for a system with disturbance ϵ. This figure is adapted from Schwenzer et al. (2021).

(b) Block diagram of RL.

Figure I.5: MPC and RL.

After investigating Bayesian system identification and state estimation, we now turn our attention to control strategies, meaning optimizing system inputs given target and

system observations. Control methods can be broadly categorized into model-based and model-free approaches. Model-based control relies on an explicit model of the system dynamics to predict future states and optimize control actions accordingly. In contrast, model-free control approaches do not require a state transition model of the system, instead learning control strategies directly from interactions with the system. A prominent example of model-based control is Model Predictive Control (MPC) (García et al., 1989; Rawlings et al., 2020).

Nominal Model Predictive Control

A to-be-controlled system is at first modeled in discrete time steps:

$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \tag{I.89a}$$

$$\boldsymbol{y}_k = h(\boldsymbol{x}_k), \tag{I.89b}$$

where $\boldsymbol{x}_k$ is the system state, $\boldsymbol{u}_k$ and $\boldsymbol{y}_k$ are the control input and system output at time step k, respectively. The function f describes the state transition, and h is the measurement function. Assuming that a full measurement or estimate of the state $\boldsymbol{x}_k$ is available, the objective of MPC is to find an optimal sequence of control inputs $\boldsymbol{u}_{k:k+N-1}$ over a finite prediction horizon N. The objective function is:

$$\min_{\boldsymbol{u}_{k:k+N-1}} \sum_{i=k}^{k+N-1} \ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) + L_f(\boldsymbol{y}_{k+N}^r, \boldsymbol{y}_{k+N}), \tag{I.90a}$$

$$\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r) = (\boldsymbol{y}_i - \boldsymbol{y}_i^r)^T Q_y (\boldsymbol{y}_i - \boldsymbol{y}_i^r) + \boldsymbol{u}_i^T Q_u \boldsymbol{u}_i, \tag{I.90b}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i), \tag{I.90c}$$

$$g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0, \quad j = 1, \ldots, n_g, \quad i = k, \ldots, k+N-1, \tag{I.90d}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = k, \ldots, k+N-1. \tag{I.90e}$$

The cost function consists of a sum of stage costs ℓ and a final cost function L_f. In the stage cost, weighting matrices Q_y and Q_u are used to balance the trade-off between tracking performance and control effort. The reference trajectory $\boldsymbol{y}^r$ represents the desired output over the prediction horizon. The constraints $g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0$ can represent physical limitations of the system or other operational constraints. A simplified schematic of MPC is shown in Figure I.5a.

The optimization problem in MPC can be solved using various numerical methods, such as sequential quadratic programming, interior-point methods, or gradient-based approaches (Diehl, 2013). The choice of method depends on the specific characteristics of the system model and constraints. The first step of the solution (Equation I.90) $\boldsymbol{u}_k^*$ is applied to

the system, and the whole process is repeated at the next time step with updated state information. That's why MPC is also known as receding horizon control (Christophersen, 2007).

Stochastic Model Predictive Control

Taking system uncertainties and disturbances into account, robust MPC (Campo and Morari, 1987) aims to satisfy system stability and constraints despite uncertainties and disturbances. There exist several sound robust MPC methods, including Min-MaxMPC (Raimondo et al., 2009), scenario MPC (Calafiore and Fagiano, 2013), and stochastic MPC (Heirung et al., 2018; Mesbah, 2016). The Min-Max MPC chooses control actions that minimize the worst possible cost under bounded disturbances. The scenario MPC samples a finite number of random uncertainty realizations as scenarios and optimizes control actions that satisfy constraints for each scenario. Stochastic MPC incorporates the probabilistic descriptions of uncertainties into a stochastic optimal control problem through defining chance constraints. A mathematical formulation of stochastic MPC is as follows (Heirung et al., 2018) :

$$\min_{\boldsymbol{u}_{k:k+N-1}} \sum_{i=k}^{k+N-1} \mathbb{E}[\ell(\boldsymbol{y}_i, \boldsymbol{u}_i, \boldsymbol{y}_i^r)] + \mathbb{E}[L_f(\boldsymbol{y}_{k+N}^r, \boldsymbol{y}_{k+N})], \tag{I.91a}$$

$$\text{subject to} \quad \boldsymbol{x}_{i+1} = f(\boldsymbol{x}_i, \boldsymbol{u}_i, \boldsymbol{w}_i), \quad \boldsymbol{y}_i = h(\boldsymbol{x}_i, \boldsymbol{v}_i), \tag{I.91b}$$

$$\mathbb{P}[g_j(\boldsymbol{x}_i, \boldsymbol{u}_i) \leq 0] \geq 1 - \beta_j, \quad \beta_j \in (0, 1], \quad j = 1, \ldots, n_g, \quad i = k, \ldots, k+N-1, \tag{I.91c}$$

$$\boldsymbol{u}_{min} \leq \boldsymbol{u}_i \leq \boldsymbol{u}_{max}, \quad i = k, \ldots, k+N-1. \tag{I.91d}$$

The chance constraints $\mathbb{P}$ regularize the inequality constraints g_i, allowing for a small probability β_j of constraint violation. The expectation in the cost function can be approximated using Monte Carlo simulation or analytical methods if the distributions are known.

2.5.4 Reinforcement Learning for Control

When an accurate model of the system is unavailable or difficult to obtain, model-free control methods can be employed. Classical model-free control methods are Proportional-Integral-Derivative (PID) controllers (Fliess and Join, 2013). In recent years, data-driven control methods attract increasing attention. Especially, Reinforcement Learning (RL) can be viewed as a model-free framework for solving optimal control problems, and has been successfully applied in various domains, including robotics, game playing, and autonomous systems.

We first introduce the terminology of RL:

- agent: the controller,
- environment: the to-be-controlled system,
- action: the control input, a_t,
- state: the complete state of the environment, s_t,
- policy: the control strategy, which can be deterministic ($a_t = \mu(s_t)$) or probabilistic ($a_t = \pi(\cdot|s_t)$),
- reward: the feedback on the agent's success, $r_t = R(s_t, a_t)$.

The foundation of RL is the Markov Decision Process (MDP) (Puterman, 1994). An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(s_{t+1}|s_t, a_t)$ is the state transition probability, $R(s_t, a_t)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The interaction process between the agent and the environment at each time step t is as follows: the agent observes the current state s_t, selects an action a_t according to its policy $\pi(a_t|s_t)$, receives a reward $r_t = R(s_t, a_t)$ from the environment, and transitions to a new state s_{t+1} based on the state transition probability $P(s_{t+1}|s_t, a_t)$. The goal of the agent is to learn an optimal policy π^* that maximizes the expected cumulative discounted reward (known as value function):

$$\pi^* = \arg\max_{\pi} \mathbb{E}\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \tag{I.92}$$

In comparison to other optimal control methods, RL methods aim to optimize long-term sequential decision-making performance. Besides, generality and scalability are the two main advantages of RL methods. They can handle nonlinear, high-dimensional, and stochastic dynamics and also non-quadratic reward functions. The work by Buşoniu et al. (2018) provides a comprehensive review of RL for control.

It is worth noting that there is also an RL method applying a surrogate model for the sake of improving the policy, which is called model-based RL (Sutton, 1990; Moerland et al., 2022). The surrogate model is learned from data in advance. Then, the model can also be used for planning, simulation, or generating synthetic data to augment real-world data. In comparison to the model-free RL, the model-based RL can reduce the required amount of samples gathered directly from the environment. From the viewpoint of control, model-based RL is also referred to learning-based control, when the surrogate model is also adaptive online. From the perspective of leveraging a state transition model, model-based RL can be viewed as a hybrid approach that combines elements of both model-based and model-free control strategies.

RL provides a flexible framework for learning control policies directly from data, enabling adaptation to changing environments and complex system dynamics. However, it often requires a large amount of data and may suffer from explainability, stability, and reliability issues. In safety-critical applications, model-based control methods are often preferred.

3 Objectives and Contributions

Starting from BayesFlow (in Section 2.4.5) (Radev et al., 2020), we endeavor to explore the potential of normalizing flows for system parameter identification, dynamics modeling, state estimation, and also optimal control. The primary objectives of this dissertation are as follows:

- To investigate the strength and limitation of BayesFlow in parameter identification;
- To extend conditional normalizing flows for parameter identification to estimate sequential system states;
- To explore the capabilities of normalizing flows as control policies.

The key contributions of this dissertation can be summarized as:

- A comprehensive examination of the original BayesFlow framework in real-world applications, assessing its performance and generalizability.
- Integration of BayesFlow with sensitivity analysis to enable parameter estimation across varying levels of identifiability.
- Development of *Stochastic BayesFlow*, which combines stochastic normalizing flows with BayesFlow to achieve robust parameter identification under uncertainty.
- Extension of conditional normalizing flows through *IMFlow*, introducing the concept of using flows as control policies.
- Proposal of *FlowSSM*, a novel method that leverages normalizing flows to model uncertainty in stochastic dynamical systems.

4 Outline

The remainder of this thesis is organized as follows:

- Chapter III outlines the methodologies developed and employed throughout this work. These methods are grouped into three categories: parameter identification, state estimation, and optimal control.
- Chapter IV presents the experimental validation of the proposed methods, structured according to the aforementioned categories.
- Chapter V summarizes the key findings of the dissertation and proposes directions for future research.

Chapter II

Literature Review

This section reviews the relevant literature about applications of deep probabilistic models in three aspects: Bayesian parameter identification; Bayesian state estimation; deep generative models as control policies. The deep probabilistic models mainly include Deep Gaussian Processs (DGPs), Bayesian Neural Networks (BNNs), Conditional Variational Autoencoders (CVAEs), Conditional Normalizing Flows (CNFs), and also conditional diffusion models. [todo: flow matching]

1 Bayesian Parameter Identification

1.1 Deep Gaussian Processes

Deep Gaussian Processs (DGPs) are sequential compositions of Gaussian processes, where the outputs of one Gaussian process are the inputs to the next. They extend the single-layer Gaussian process regression (GPR) and can represent compositional or hierarchical structure of variables. A survey (Jakkala, 2021) summarizes the improved methods of DGPs.

Abraham and Deo (2023) proposed a DGP-based method for Bayesian inference of compositional parameters from Partial Differential Equations (PDEs). The parameters are assigned with a DGP prior which models compositional functions. Then, the posterior distribution of the parameters is optimized via variational inference.

Applications of DGPs for Bayesian inference span multiple domains. For instance, Park et al. (2018) utilized DGPs to identify contaminant release sources, while Desai et al. (2023) employed them for air quality estimation.

In biomedical contexts, Alaa and van der Schaar (2017) leveraged DGPs to assess patient survival under competing risks.

These studies show promising results in comparison to GPRs as well as other machine learning methods, and highlight the versatility and effectiveness of DGPs in addressing parameter inference challenges. However, the computational cost and storage cost remain the main drawbacks of DGPs, especially for high-dimensional data.

1.2 Bayesian Neural Networks

Applying Bayesian Neural Networks (BNNs) for Bayesian inference is usually combined with other methods for better extracting information from observation data. For example, Yang et al. (2021) integrated a BNN with Physics-Informed Neural Networks (PINNs) (B-PINN) to infer parameters in Partial Differential Equations (PDEs) with Gaussian noise. PINNs construct a neural network to approximate the forward solution to the underlying ordinary differential equation (ODE) or PDE problem by minimizing reconstruction loss of data, while regularized by the underlying governing ODE or PDE residual loss. Meanwhile, model parameters can also be estimated probabilistically via Markov Chain Monte Carlo (MCMC) or variational inference. The B-PINN achieved more accurate results with stronger noise in observations.

Merkatas and Särkkä (2023) used an autoregressive BNN for estimating the system parameters, together with a nonparametric noise model to capture complex noise characteristics in the data. The BNN network parameters are approximated through Hamiltonian Monte Carlo (HMC) sampling approach. BNNs are flexible to be integrated with other models and can provide uncertainty quantification, at the cost of higher computational complexity. On the other hand, applying BNNs for Bayesian inference is not straightforward and must be combined with other architectures. Besides, the uncertainty of network weights does not necessarily translate to the uncertainty of the target variables.

1.3 Conditional Variational Autoencoders

Conditional Variational Autoencoders (CVAEs) represent a conditional extension of the Variational Autoencoder (VAE) framework. CVAEs have been widely adopted for amortized Bayesian inference across various scientific domains. For example, Nautiyal et al. (2024) introduced a CVAE-based approach for efficient Bayesian inference in scientific simulators.

In astronomy, CVAEs have enabled Bayesian parameter estimation from gravitational wave data, as demonstrated by Yamamoto and Tanaka (2020), Gabbard et al. (2022), and Sun et al. (2025). Additionally, Ferreira and Bejger (2025) applied CVAEs to infer the equation of state of neutron stars.

In biomedical engineering, CVAEs have been used for neuroimaging analysis under varying correlated parameters (2025), and for parameter estimation in mouse cardiac muscle studies (2025).

These examples highlight the versatility and effectiveness of CVAEs for Bayesian inference of high-dimensional data. One shortcoming exhibited in the research (Sun et al., 2025) is that the CVAE tends to estimate broader posterior distributions compared to other methods, indicating a potential limitation in capturing precise uncertainty.

1.4 Conditional Normalizing Flows

BayesFlow (in Section 2.4.5) (Radev et al., 2020) provides a general framework for amortized Bayesian inference using CNFs. Besides our work, it has also been applied to various scientific domains. For instance, Kucharský and Bürkner (2025) applied BayesFlow in multiple cognitive models. Zeng et al. (2023) demonstrated its effectiveness in damage detection in structural health monitoring. Rahmandad et al. (2025) utilized BayesFlow to infer parameters from system dynamics, such as the random walk model.

Some other works share similar idea with BayesFlow. De Santi et al. (2024) referred the summary network as an embedding neural network. Zhdanov et al. (2022) extended the condition with richer synthetic data, which is generated by a CVAE. Dong et al. (2025) arranged a CNF through taking the conditioning variable as an additional input to each coupling layer and enlarging the latent space as well.

These works demonstrate the effectiveness of CNFs for amortized Bayesian inference. Especially, the design of the summary network needs to be tailored to the specific characteristics of the data, which is crucial for extracting informative representations from observations and inferring parameters precisely.

1.5 Conditional Diffusion Models

Diffusion models have recently emerged as powerful tools for Bayesian inference due to their ability to model complex, multimodal distributions. Zhu et al. (2025) introduced a diffusion-based framework for parameter identification in dynamic power systems, demonstrating improved accuracy over traditional methods.

Conditional diffusion models have also been applied to infer parameters, unobservable states, and system dynamics in PDEs (Haitsiukevich et al., 2024).

Furthermore, Baldassari et al. (2023) developed a framework for conditional score-based diffusion models in infinite-dimensional function spaces to perform Bayesian inference for linear inverse problems.

Chen et al. (2025) proposed a conditional diffusion model for amortized posterior estimation, showing superior performance compared to CNFs, particularly in reconstructing multimodal distributions and capturing variable boundaries. Nevertheless, the computational cost of inference with diffusion models is much higher than other Deep Generative Models (DGMs) due to its iterative sampling procedure.

2 Bayesian State Estimation

After introducing versatile applications of deep probabilistic models for Bayesian parameter identification, this section focuses on the applications of deep probabilistic approaches for

state estimation, including data-assisted Bayesian Filters and deep state space model.

2.1 Data-Assisted Bayesian Filters

From the section 2.5.2 we have known that estimating state from the noisy observations is strongly dependent on the completeness of system models and strong assumptions on noise. Under model mismatches or incompleteness, deep probabilistic alternatives have been proposed to integrate Bayesian filters with offline data. The data-assisted Bayesian filters take advantage of the rigor of the state prediction and update, achieving explainability and better results than solely data-driven methods.

For instance, Yan et al. (2024) introduced a Bayesian gated recurrent framework for non-Markov dynamical systems, which consists of a memory update gate for capturing the temporal regularities in state evolution, a state prediction gate with the evolution mismatch compensation, and a state update gate with the observation mismatch compensation; it achieved stronger state estimation performance than standard Recurrent Neural Networks (RNNs) and Transformer baselines.

Fraccaro et al. (2017) proposed the Kalman VAE, which combines a VAE with a linear Gaussian state-space model. The VAE encoder learns a low-dimensional latent representation of the observations, while the Kalman filter performs recursive Bayesian state estimation in the latent space. The decoder reconstructs the observations from filtered latent states, allowing uncertainty-aware tracking of temporal dynamics. Similar ideas have been explored in (Pasmans et al., 2025; Zhang et al., 2023).

Normalizing Flow (NF) have also been explored to be combined with linear Gaussian state-space model. Wang et al. (2025) utilized a NF to transform nonlinear model with non-Gaussian noise into latent space, where a Kalman filter is operated. The major advantage in NFs compared with VAEs lies in the invertibility, which allows efficient sampling of target filtering distribution in the original space. The authors shown that the flow-based Bayesian filter achieves better accuracy and computational efficiency compared to conventional filtering methods in challenging high-dimensional settings.

Another way of applying NF is proposed by Chen et al. (2021). A CNF is employed to construct flexible proposal distributions in differentiable particle filters. The NF is conditioned on the observed data to migrate particles to areas closer to the true posterior. The integration of CNFs improved the accuracy in a visual tracking task.

2.2 Deep State-Space Models

Constructing the state transition and observation models in State-Space Models (SSMs) using neural networks allows for the representation of a wide range of dynamic behaviors, thanks to their flexibility and expressive capability. The review (Lin and Michailidis,

2024) provides a selective review of recent advancements in deep neural network-based approaches for SSMs, and presents a unified perspective for discrete time deep state space models and continuous time ones such as latent neural ODEs and neural Stochastic Differential Equations (SDEs). Recent advancements in sequence modeling have led to the emergence of structured SSMs as an efficient alternative to RNNs and Transformers, addressing challenges in long-range dependency modeling and computational efficiency. A comprehensive survey about structured SSMs is conducted by Somvanshi et al. (2025).

Similar to the work of Wang et al. (2025), leveraging NFs to model noises in the SSMs also enables fast and accurate system dynamics learning, high-dimensional state estimation, and time-series forecasting. Lin et al. (2025) proposed an efficient transformed Gaussian process SSM that integrates a single shared Gaussian Process (GP) with input-dependent NFs, yielding an expressive implicit process prior that captures complex, non-stationary transition dynamics while significantly reducing model complexity.

3 Deep Probabilistic Models and Time Series Modeling

We also review the combinations of deep probabilistic models with time series modeling architectures, which provide us inspirations to extend normalizing flows in state estimation tasks. Besides, we summarize the applications of normalizing flows for modeling stochastic processes, which is closely related to our proposed method FlowSSM in Chapter III.

3.1 Integration of Deep Probabilistic Models with Time Series Modeling Architectures

Identification of nonlinear system dynamics solely from input-output data is a challenging task with solely time series modeling methods. In contrast, deep probabilistic models, especially DGMs, have shown great potential in capturing complex nonlinear relationships while providing uncertainty quantification. The combinations of both methods have been explored in recent years.

Brusaferri et al. (2019) stacked Long Short-Term Memory (LSTM) layers with a probabilistic layer, obtaining a Bayesian inference model for nonlinear system identification, capturing nonlinear dynamics while providing uncertainty quantification.

Paniagua and López (2024) introduced a framework integrating a VAE with Nonlinear Autoregressive eXogenous (NARX) for identifying nonlinear systems using input/output data. The VAE encodes the input/output data into a latent space, and the NARX decodes the latent space information and reconstructs the system outputs. This approach effectively captures nonlinear relationships effectively and improves the robustness through the VAE in comparison to the NARX alone.

Roeder et al. (2019) proposed an amortized Bayesian inference method for learning hierarchical nonlinear dynamical systems. The method extends the VAE by incorporating the structural information of the system as hierarchical conditions. The encoder performs amortized inference over latent variables conditioned on observed trajectories and system structure. The decoder employs differentiable ODEs to describe the system dynamics, which can include both known mechanistic components and learned neural components. This method enables integrating knowledge of system structure with data-driven learning, improving interpretability and generalization.

3.2 Normalizing Flows and Stochastic Processes

Another viewpoint of probabilistic time series modeling is to consider it as learning the underlying stochastic process that generates the data. In this section, we focus on the research about leveraging NFs to model stochastic processes.

Stochastic Continuous Normalizing Flow (SCNF) (Hodgkinson et al., 2021) treats SDEs as random ODEs. Random ODEs are a type of differential equation where the vector field includes a stochastic process, which is approximated by deterministic functions. Random ODEs are trained in the manner of continuous normalizing flows. However, SCNF suffers from the overfitting issue of the vanishing diffusion term.

Continuous Time Flow Process (CTFP) (Deng et al., 2021b) models stochastic processes by transforming latent Wiener processes via continuous normalizing flows. The continuous normalizing flows are trained in the forward direction through mapping the data samples to the latent variables. Time-Changed Normalizing Flow (Bekri et al., 2024) extends the base Wiener process in CTFP with a time function. Continuous Latent Process Flow (Deng et al., 2021a), as an improvement of CTFP, applies a LatentSDE (Li et al., 2020) as the encoder to generate latent variables, and a continuous normalizing flow as the decoder. Although these methods shown good performance in generating probabilistic time series, they are computationally expensive and not able to solve initial value problems directly or model dynamical systems with control inputs, which makes them unsuitable for many engineering applications.

Although several works have demonstrated the potential of normalizing flows for modeling stochastic processes, their application to system dynamics modeling and state estimation remains limited. Key challenges include high computational cost, difficulty handling control inputs and initial-value problems, and risks of overfitting or vanishing diffusion. Addressing these issues represents a promising direction for our research.

4 Deep Generative Models as Control Policy

Moreover, deep generative approaches for capturing input-output relationships are well-suited for integration into model-free Reinforcement Learning (RL) frameworks. The modeled uncertainty plays a crucial role in enabling robust decision-making. A survey (Zhu et al., 2024) provides a comprehensive overview of recent research efforts focused on the application of diffusion models within the domain of reinforcement learning. Diffusion models demonstrate significant flexibility and capacity as general-purpose tools for planning, policy generation, and data synthesis. The recent convergence with powerful Large Language Models (LLMs) has significantly enhanced this capability. This integration unlocks the multi-modal strength of diffusion models, enabling them to tackle diverse and complex tasks in robotics (Wen et al., 2025).

However, in specific single-task scenarios, NFs may prove more suitable. NFs often offer a superior balance between computational cost and the strength of accurate density estimation. This is particularly required in domains like physics-based modeling, where the underlying noise distribution is often less complex than in general image/language generation tasks. Especially, Ghugare and Eysenbach (2025) brought up that the probabilistic modeling and inference capability of NFs suits better for RL framework in comparison to diffusion models, including acting as a policy, as a Q-function or value model, and for modeling state densities.

In the following, we summarize different RL-methods equipped with NF or Flow Matching (FM).

Table II.1: Normalizing Flows in Reinforcement Learning

Flow Model	Role in RL	Main Contribution
Discrete NF (Ward et al., 2019; Mazoure et al., 2020)	Policy of SAC	Improved expressiveness of the policy distribution and efficient exploration of sparse reward.
Neural Spline Flow (Lind et al., 2025)	Visuomotor Policy	Comparable performance to Diffusion Policy and improved sample efficiency in training while up 30x faster in inference.
Neural Spline Flow (Chang et al., 2022)	Expert policy of inverse RL	Achieving good recovery of the true reward signal and imitation from observation.

Continued on next page...

Table II.1: Normalizing Flows in RL (Continuation)

Flow Model	Role in RL	Main Contribution
Causal NF (Cho et al., 2024)	Learning transition dynamics and reward function in offline policy training	Improved generalization in sequential decision-making tasks, revealing a high potential for OoD adaptation in offline RL.
Coupling Flow (Khader et al., 2021; Yin et al., 2022)	Diffeomorphic transfomration from state to action (policy)	Application in complex continuous control tasks while ensuring stability.
Coupling Flow (Rietz et al., 2024)	Constrained policy of SAC	Ensuring constraints satisfactory over reward engineering while offering enhanced interpretability.
Coupling Flow (Tarasov et al., 2025)	Policy in VLA models	Comparable performance to Diffusion Policy and improved parameter efficiency while faster in inference.
RealNVP (Brahmanage et al., 2023)	Policy of DDPG	Significant reduction of constraint violations.
RealNVP (Power and Berenson, 2022)	Learning control policy in MPC	FlowMPPI with projection outperforms state-of-the-art MPC baselines on both in-distribution and OoD environments.
RealNVP (Boborzi et al., 2021)	Policy of DAC	Enabling imitation of complex continuous control policies.
Autoregressive Flow (Agarwal et al., 2022)	Motion planner in IL	Achieving good performance in real-world autonomous driving tasks.
RealNVP (Ma et al., 2020)	Policy representation	Capturing multi-modal action distributions.
RealNVP (Lee and Diwan, 2023)	Policy gradient in model-based RL	Capturing multi-modal action distributions.
MAF and Glow (Urain et al., 2020)	Learning dynamics model	Improved representation of stochastic dynamics while ensuring stability.
MAF (Freund et al., 2023)	Distribution matching in IL	Better capturing state-action distribution.
FM (Pivezhandi and Saifullah, 2025)	Augmenting data	Achieving stable convergence in model-based RL.

Continued on next page...

Table II.1: Normalizing Flows in RL (Continuation)

Flow Model	Role in RL	Main Contribution
FM (Xu and Liu, 2024)	Generating trajectories	Outperforming other baselines in continuous and discrete environments with stochastic dynamics.
FM (Lv et al., 2025)	Optimal-behavior constrained policy in online RL	Regularization of learned policy through implicit guidance mechanism; achieving stable training and improved sample efficiency.
FM (Zhang et al., 2025d)	Policy of SAC	Improved performance and sample efficiency in continuous control tasks.
FM (Kong et al., 2025)	Estimating dynamics gap	Improved generalization for learning online dynamics; Outperforming SotA-RL baselines with shifted dynamics.
FM (Zhang et al., 2025a)	Policy in IL	Fast policy generation based on 3D vision; Achieving improved success rates and faster inference time for robotic manipulation.
FM (Rouxel et al., 2025)	Goal conditioned policy	Estimating distributional bounds.
FM (McAllister et al., 2025)	Policy in PGRL	Capturing multimodal action distributions in continuous control tasks.
FM (Wan et al., 2025)	Student & teacher policy in IL	Improved generalization of learned policy and the robustness against sub-optimal expert data.
FM (Zhong et al., 2025)	Modeling value distribution (Critic)	Reduction of gradient variance.
Energy-based NF (Chao et al., 2024)	Policy of Maximum Entropy RL	Integration of the policy evaluation steps with the policy improvement steps, resulting in a single objective training process.
Energy-guided FM (Alles et al., 2025)	Policy for offline RL	Competitive performance and constant training time in flow sampling steps.
Energy-guided FM (Zhang et al., 2025b)	Policy in offline RL	Faster inference time.
Rectified Flow (FM) (Zhang et al., 2025c)	Policy in online RL	Surpassing diffusion policy in continuous robotic control.

Chapter III

Methodologies

1 Parameter Identification

As we have introduced in Section 2.3, different deep probabilistic methods are proposed for Bayesian parameter identification. In real applications, it is usually desired to identify the system parameters from multimodal observations. We start to introduce a universal framework with deep generative models for amortized Bayesian inference. This framework can be seen as a generalization of the BayesFlow (Radev et al., 2020).

1.1 Universal Framework with Deep Generative Models for Amortized Bayesian Inference

Radev et al. (2020) proposed BayesFlow as an amortized Bayesian inference approach with normalizing flows. BayesFlow consists of a summary network for extracting information from the observations, and a conditional normalizing flow for learning the posterior distribution of parameters conditioned on the extracted information. This framework can be generalized to other deep generative models, such as Variational Autoencoder (VAE), Flow Matching (FM), and diffusion models. Figure III.1 illustrates the framework. We call the summary network here "condition network" to emphasize that the embeddings of observations function as condition in the Deep Generative Models (DGMs). The major advantages of applying deep generative models as the Bayesian inference backbone are threefold:

- DGMs consider the parameter identification as posterior inference, which provides naturally trustworthiness of the estimates;
- DGMs seek to model the posterior distribution on a large dataset, which is suitable for amortized Bayesian inference;
- compared with other architectures, like Bayesian Neural Networks (BNNs), DGMs achieve preciser posterior inference because of the separate inference and generation procedures.

In many generative tasks, such as text, image, and video generation, the condition is assumed to be simple. In contrast, for solving inverse problems, such as parameter identi-

fication, the generative model is supposed to "generate" variables under some complexer conditions. Thus, a well-designed condition network is essential to extract most informative features from the observations. This universal framework automatically encourages the condition network to deliver desirable embeddings to the deep generative model. Besides, each observation is only required to processed once while sampling the trained generative model for multiple times, which is computationally efficient.

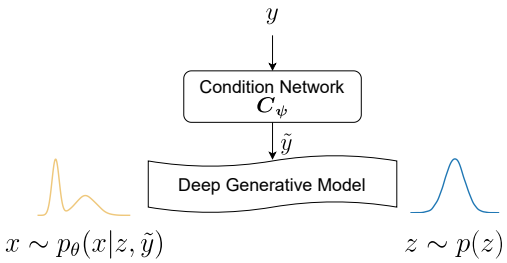


Figure III.1: Universal Framework of Using Deep Generative Models for Amortized Bayesian Inference. The corresponding algorithm is shown on the right side (Algorithm 1). Training paradigms differ significantly across DGMs. While VAEs and Normalizing Flows (NFs) maximize the Evidence Lower Bound (ELBO) and log-likelihood respectively, flow-based and diffusion models typically minimize a matching cost—regressing either a vector field (FMs), a score function, or added noise (Denoising Diffusion Probabilistic Models (DDPM)). Consequently, Bayesian inference is performed either via a direct deterministic mapping or by numerically integrating a differential equation (ODE or SDE).

Algorithm 1 DGM-ABI Training & Inference

Require: Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{M}$
Require: Prior $p(z)$, Condition network C_ψ
Require: Generator $p_\theta(x \mid z, \tilde{y})$, (Optional: Inference $q_\phi(z \mid x, \tilde{y})$)

Stage 1: Amortized Learning
while not converged **do**
  Sample batch $\{(x, y)\} \sim \mathcal{D}$
  Compute embeddings: $\tilde{y} \leftarrow C_\psi(y)$
  Sample latent: $z \sim p(z)$
  Compute objective gradient:
  $g \leftarrow \nabla_{\theta,\psi,\phi} \mathcal{L}$
  Update parameters: $(\theta, \psi, \phi) \leftarrow (\theta, \psi, \phi) - \eta \cdot g$
end while
Stage 2: Bayesian Inference
Given observed condition y_{obs}:
$\tilde{y}_{\text{obs}} \leftarrow C_\psi(y_{\text{obs}})$
for $k = 1$ to K **do**
  Sample $z^{(k)} \sim p(z)$
  Generate $x^{(k)} \sim p_\theta(x \mid z^{(k)}, \tilde{y}_{\text{obs}})$
end for
return Posterior samples $\{x^{(k)}\}_{k=1}^{K}$

1.2 Normalizing Flows for Parameter Identification

Why are NFs more suitable for parameter identification tasks given time series observations? In this section, we will answer this question in terms of (1) inference accuracy and precision (2) reliability, i.e., whether the ground truth lies in the posterior, (3) speed including training convergence time and computational time, (4) complexity, i.e., handling multimodal distributions, (5) correlation, i.e., identifying linearly correlated parameters, (6) sample efficiency, i.e., posterior contraction while increasing data amount.

1.3 Stochastic Normalizing Flows

Normalizing flows, including flow matching, are deterministic by design, which enables precise and efficient density estimation. However, in practice this deterministic structure can limit their ability to fully capture complex, highly multimodal target distributions, especially when model capacity or data are limited. One way to alleviate this limitation is to combine normalizing flows with Markov Chain Monte Carlo (MCMC) methods. In such hybrid approaches, the normalizing flow serves as an expressive proposal or initialization mechanism, while MCMC refines the samples by enforcing consistency with the true posterior through its accept–reject mechanism or transition kernel. The stochastic exploration of MCMC improves mode coverage and helps the overall system discover and accurately represent multiple modes that might be underrepresented by the deterministic flow alone. Moreover, by repeatedly evaluating the forward dynamics within the MCMC procedure, the method can better discriminate between parameter configurations that produce similar noisy observations, thereby improving parameter identifiability under measurement noise.

Wu et al. (2020) proposed to construct Stochastic Normalizing Flows (SNFs) by alternating a sequence of deterministic Invertible Neural Networks (INNs) and stochastic MCMC sampling layers (INN - MCMC - $\cdots$ - INN - MCMC). Hagemann et al. (2022) provided theoretical foundation for stochastic normalizing flows and proved their consistency to Conditional Normalizing Flows (CNFs) for solving inverse problems. An SNF is a pair of Markov chains of random variables $((U_0, \ldots, U_T), (V_T, \ldots, V_0))$. The transformation between two adjacent variables is either a deterministic INN block or a stochastic MCMC sampling layer. Denoting the process $(U_t)_{t=0}^T$ as the forward process, U_0 represents the data and U_T is trained to follow a standard normal distribution. Conversely, in the inverse process $(V_t)_{t=0}^T$, V_T is sampled from the standard normal distribution and V_0 follows the target density. Because both processes are Markov chains, the transformations can be written as conditional distributions, where each state depends only on its immediate predecessor. We denote the transformations in the forward process by $\mathcal{T}_t = P(U_t \mid U_{t-1})$ and those in the inverse process by $\mathcal{T}_t^{-1} = P(V_{t-1} \mid V_t)$. An illustration of such an SNF architecture is shown in Figure III.2.

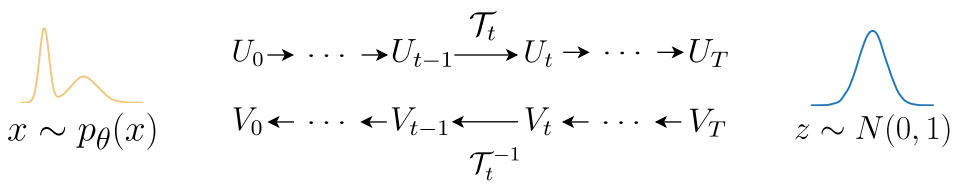


Figure III.2: The architecture of an SNF.

Unlike CNFs, for which the objective function is minimizing the KL-divergence between the learnt and target density, the objective function for training an SNF is to minimize the difference of the joint probabilities between the forward and inverse processes. We define the joint probability of forward process as $p_\theta(u_{0:T}) = p_X(u_0)\Pi_{t=1}^T p_\theta(u_t|u_{t-1})$, while the inverse process as $q_\theta(v_{0:T}) = p_Z(v_T)\Pi_{t=1}^T q_\theta(v_{t-1}|v_t)$. Same as CNFs, the objective function is trained in the forward direction.

$$\begin{aligned}\mathcal{L}_{SNF}(\theta) = D_{\mathrm{KL}}(p_\theta(u_{0:T})||q_\theta(v_{0:T})) &= \mathbb{E}_{u_{0:T}\sim p_\theta}[\log p_\theta(u_{0:T}) - \log q_\theta(v_{0:T})] \\ &= \mathbb{E}_{u_{0:T}\sim p_\theta}[-\log p_Z(u_T) + \log p_X(u_0) + \sum_{t=1}^{T} \log \frac{p_\theta(u_t|u_{t-1})}{q_\theta(u_{t-1}|u_t)}]. \quad \text{(III.1)}\end{aligned}$$

The critical point is the calculation of the term $[p_\theta(u_t|u_{t-1})/q_\theta(u_{t-1}|u_t)]$.

- For a deterministic transformation, the term is equivalent to the change of variable, which is given by:

$$\frac{p_\theta(u_t|u_{t-1})}{q_\theta(u_{t-1}|u_t)} = \frac{du_{t-1}}{du_t} = -\log\det|J_f(u_{t-1})|\,. \quad \text{(III.2)}$$

- For a stochastic transformation, the term is the KL-divergence, measuring how well the backward kernel inverts the forward one at each step. The stochastic transformation is supposed to be invertible and has to fulfill the detailed balance condition with respect to some density p_t, which is given by:

$$p_t(u_t)q_\theta(u_{t-1}|u_t) = p_t(u_{t-1})p_\theta(u_t|u_{t-1}). \quad \text{(III.3)}$$

  The density p_t is the target density of the stochastic layer at the t-th step. It is chosen to be an interpolation between p_Z and p_X, which is given by:

$$p_t = c \cdot p_Z^{t/T} \cdot p_X^{(T-t)/T}. \quad \text{(III.4)}$$

  The parameter c is a normalizing constant.

  For computational efficiency, the stochastic transformation is chosen to be an Metropolis-adjusted Langevin algorithm (MALA) (See in Section2.2.2) kernel. Then, the term $[p_\theta(u_t|u_{t-1})/q_\theta(u_{t-1}|u_t)]$ is given by:

$$\frac{p_\theta(u_t|u_{t-1})}{q_\theta(u_{t-1}|u_t)} = \frac{p_t(u_t)}{p_t(u_{t-1})} = \exp\left(-\frac{1}{2\tau}(\|\eta_t\|_2^2 - \|\tilde{\eta}_t\|_2^2)\right). \quad \text{(III.5)}$$

where

$$\eta_t = u_t - u_{t-1} + \frac{\sqrt{\tau}}{2}\nabla E_t(u_{t-1}), \quad \tilde{\eta}_t = u_{t-1} - u_t + \frac{\sqrt{\tau}}{2}\nabla E_t(u_t). \tag{III.6}$$

1.4 Relationship Between Sensitivity Analysis

The parameter identification from time series observations is consistent with global sensitivity analysis.

1.5 When Does the Framework Fail?

The framework of amortized Bayesian inference using generative models is highly dependent on the embedded information extracted by the condition network. When the time series are noisy, dynamic, or stochastic. The information can not be well extracted.

2 State Estimation

IMFlow ability limited

limitation for bayesflow for identifying highly dynamical data

transformer-bayesian neural network better for identification of battery state-of-health

FlowSSM

3 Optimal Control

Model free control with IMFlow

Chapter IV

Experiments and Results

1 Overview

2 Parameter Identification

von der pol single track car quadcopter

3 State Estimation

4 Optimal Control

Chapter V

Discussion and Conclusion

Abstract

Zusammenfassung

Own Contributions

This chapter gives an overview of my contributions in distinction to team efforts.

Own share in data acquisition and data analysis

Own share in Data Acquisition.

Own share in Data Analysis.

Own Publications

In this section, all publications that I was a part of and contributed to during my Ph.D. work are listed. It is subdivided into *First Authorships* and *Co-Authorships*.

First Authorships - Peer Reviewed International Journal Publications

Y. Zhang, L. Mikelsons. **Advanced Modeling and Simulation in Engineering and Science**. *Sensitivity-guided iterative parameter identification and data generation with BayesFlow and PELS-VAE for model calibration.* doi:10.1186/s40323-023-00246-y.

First Authorships - Peer Reviewed International Conference Publications

Y. Zhang, L. Mikelsons. **2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)**. *Solving Stochastic Inverse Problems with Stochastic BayesFlow.* doi:10.1109/AIM46323.2023.10196190.

Y. Zhang, L. Mikelsons. **2024 IEEE 63rd Conference on Decision and Control(CDC)**. *Inverse Modeling with Conditional Normalizing Flows for Data-Driven Model Predictive Control*. doi:10.1109/CDC56724.2024.10886862.

Co-Authorships

M.E. Heringhaus, Y. Zhang, A. Zimmermann, L. Mikelsons. **Sensors 2022**. *Towards Reliable Parameter Extraction in MEMS Final Module Testing Using Bayesian Inference*. doi:10.3390/s22145408.

Bibliography

Kweku Abraham and Neil Deo. Deep gaussian process priors for bayesian inference in nonlinear inverse problems, 2023. URL https://arxiv.org/abs/2312.14294.

Ryan P. Adams, Jeffrey Pennington, Matthew J. Johnson, Jamie Smith, Yaniv Ovadia, Brian Patton, and James Saunderson. Estimating the spectral density of large implicit matrices, 2018. URL https://arxiv.org/abs/1802.03451.

Shubhankar Agarwal, Harshit Sikchi, Cole Gulino, Eric Wilkinson, and Shivam Gautam. Imitative planning using conditional normalizing flow, 2022. URL https://arxiv.org/abs/2007.16162.

Ahmed M. Alaa and Mihaela van der Schaar. Deep multi-task gaussian processes for survival analysis with competing risks. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 2326–2334, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Edward Allen. *Modeling with itô stochastic differential equations*, volume 22. Springer, 2007.

Marvin Alles, Nutan Chen, Patrick van der Smagt, and Botond Cseke. Flowq: Energy-guided flow policies for offline reinforcement learning, 2025. URL https://arxiv.org/abs/2505.14139.

Govinda Anantha Padmanabha and Nicholas Zabaras. Solving inverse problems using conditional invertible neural networks. *Journal of Computational Physics*, 433:110194, 2021. ISSN 0021-9991. doi: https://doi.org/10.1016/j.jcp.2021.110194. URL https://www.sciencedirect.com/science/article/pii/S0021999121000899.

Lynton Ardizzone, Jakob Kruse, Sebastian Wirkert, Daniel Rahner, Eric W. Pellegrini, Ralf S. Klessen, Lena Maier-Hein, Carsten Rother, and Ullrich Köthe. Analyzing inverse

problems with invertible neural networks, 2019a. URL https://arxiv.org/abs/1808.04730.

Lynton Ardizzone, Carsten Lüth, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Guided image generation with conditional invertible neural networks, 2019b. URL https://arxiv.org/abs/1907.02392.

Andrea Asperti. Variance loss in variational autoencoders, 2020. URL https://arxiv.org/abs/2002.09860.

Lorenzo Baldassari, Ali Siahkoohi, Josselin Garnier, Knut Solna, and Maarten V. de Hoop. Conditional score-based diffusion models for bayesian inference in infinite dimensions, 2023. URL https://arxiv.org/abs/2305.19147.

A.L. Barker, D.E. Brown, and W.N. Martin. Bayesian estimation and the kalman filter. *Computers & Mathematics with Applications*, 30(10):55–77, 1995. ISSN 0898-1221. doi: https://doi.org/10.1016/0898-1221(95)00156-S. URL https://www.sciencedirect.com/science/article/pii/089812219500156S.

Jens Behrmann, Will Grathwohl, Ricky T. Q. Chen, David Duvenaud, and Jörn-Henrik Jacobsen. Invertible residual networks, 2019. URL https://arxiv.org/abs/1811.00995.

Naoufal El Bekri, Lucas Drumetz, and Franck Vermet. Time changed normalizing flows for accurate sde modeling. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 6395–6399. IEEE, April 2024. doi: 10.1109/icassp48485.2024.10446131.

Julian Besag. Markov chain monte carlo methods for statistical inference. *Department of Statistics University of Washington, Seattle, USA*, 2004.

Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006. ISBN 0387310738.

L. Bittner. L. s. pontryagin, v. g. boltyanskii, r. v. gamkrelidze, e. f. mishechenko, the mathematical theory of optimal processes. viii + 360 s. new york/london 1962. john wiley & sons. preis 90/–. *ZAMM - Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik*, 43(10–11):514–515, January 1963. ISSN 1521-4001. doi: 10.1002/zamm.19630431023. URL http://dx.doi.org/10.1002/zamm.19630431023.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, April 2017. ISSN 1537-274X. doi: 10.1080/01621459.2017.1285773. URL `http://dx.doi.org/10.1080/01621459.2017.1285773`.

Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks, 2015. URL `https://arxiv.org/abs/1505.05424`.

Damian Boborzi, Christoph-Nikolas Straehle, Jens S. Buchner, and Lars Mikelsons. Learning normalizing flow policies based on highway demonstrations. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 22–29, 2021. doi: 10.1109/ITSC48978.2021.9564456.

Léon Bottou. *On-line learning and stochastic approximations*, page 9–42. Cambridge University Press, USA, 1999. ISBN 0521652634.

Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In Stefan Riezler and Yoav Goldberg, editors, *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1002. URL `https://aclanthology.org/K16-1002/`.

Janaka Brahmanage, Jiajing LING, and Akshat Kumar. Flowpg: Action-constrained policy gradient with normalizing flows. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 20118–20132. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/3fd9fe8ec6d7238bf71784797399bb61-Paper-Conference.pdf`.

Alessandro Brusaferri, Matteo Matteucci, Pietro Portolani, and Stefano Spinelli. Nonlinear system identification using a recurrent network in a bayesian framework. In *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*, volume 1, pages 319–324, 2019. doi: 10.1109/INDIN41052.2019.8972113.

Lucian Buşoniu, Tim de Bruin, Domagoj Tolić, Jens Kober, and Ivana Palunko. Reinforcement learning for control: Performance, stability, and deep approximators. *Annual Reviews in Control*, 46:8–28, 2018. ISSN 1367-5788. doi: https://doi.org/10.1016/j.arcontrol.2018.09.005. URL `https://www.sciencedirect.com/science/article/pii/S1367578818301184`.

Giuseppe C. Calafiore and Lorenzo Fagiano. Robust model predictive control via scenario optimization. *IEEE Transactions on Automatic Control*, 58(1):219–224, January 2013. ISSN 1558-2523. doi: 10.1109/tac.2012.2203054. URL http://dx.doi.org/10.1109/TAC.2012.2203054.

Peter J. Campo and Manfred Morari. Robust model predictive control. In *1987 American Control Conference*, pages 1021–1026, 1987. doi: 10.23919/ACC.1987.4789462.

Wei-Di Chang, Juan Camilo Gamboa Higuera, Scott Fujimoto, David Meger, and Gregory Dudek. Il-flow: Imitation learning from observation using normalizing flows, 2022. URL https://arxiv.org/abs/2205.09251.

Chen-Hao Chao, Chien Feng, Wei-Fang Sun, Cheng-Kuang Lee, Simon See, and Chun-Yi Lee. Maximum entropy reinforcement learning via energy-based normalizing flow. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 56136–56165. Curran Associates, Inc., 2024. doi: 10.52202/079017-1786. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/661c37f3b098bdee53fd7d9c4ef6964a-Paper-Conference.pdf.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations, 2019. URL https://arxiv.org/abs/1806.07366.

Ricky T. Q. Chen, Jens Behrmann, David Duvenaud, and Jörn-Henrik Jacobsen. Residual flows for invertible generative modeling, 2020. URL https://arxiv.org/abs/1906.02735.

Tianyu Chen, Vansh Bansal, and James G. Scott. Conditional diffusions for amortized neural posterior estimation, 2025. URL https://arxiv.org/abs/2410.19105.

Xiongjie Chen, Hao Wen, and Yunpeng Li. Differentiable particle filters through conditional normalizing flow, 2021. URL https://arxiv.org/abs/2107.00488.

Zhe Chen. Bayesian Filtering: From Kalman Filters to Particle Filters, and Beyond. *Statistics*, 182, January 2003. doi: 10/bkztpw. 01234 ZSCC: 0001234.

Minjae Cho, Jonathan P. How, and Chuangchuang Sun. Out-of-distribution adaptation in offline rl: Counterfactual reasoning via causal normalizing flows, 2024. URL https://arxiv.org/abs/2405.03892.

Frank J. Christophersen. *Receding Horizon Control*, pages 26–38. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. ISBN 978-3-540-72701-9. doi: 10.1007/978-3-540-72701-9_3. URL https://doi.org/10.1007/978-3-540-72701-9_3.

Andreas C. Damianou and Neil D. Lawrence. Deep gaussian processes, 2013. URL https://arxiv.org/abs/1211.0358.

Federico De Santi, Massimiliano Razzano, Francesco Fidecaro, Luca Muccillo, Lucia Papalini, and Barbara Patricelli. Deep learning to detect gravitational waves from binary close encounters: Fast parameter estimation using normalizing flows. *Physical Review D*, 109(10), May 2024. ISSN 2470-0029. doi: 10.1103/physrevd.109.102004. URL http://dx.doi.org/10.1103/PhysRevD.109.102004.

A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society. Series B (Methodological)*, 39(1):1–38, 2025/10/08/ 1977. ISSN 00359246. URL http://www.jstor.org/stable/2984875. Full publication date: 1977.

Ruizhi Deng, Marcus A. Brubaker, Greg Mori, and Andreas M. Lehrmann. Continuous latent process flows, 2021a.

Ruizhi Deng, Bo Chang, Marcus A. Brubaker, Greg Mori, and Andreas Lehrmann. Modeling continuous stochastic processes with dynamic normalizing flows, 2021b.

Aadesh Desai, Eshan Gujarathi, Saagar Parikh, Sachin Yadav, Zeel Patel, and Nipun Batra. Deep gaussian processes for air quality inference. In *Proceedings of the 6th Joint International Conference on Data Science & Management of Data (10th ACM IKDD CODS and 28th COMAD)*, CODS-COMAD '23, page 278–279, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450397971. doi: 10.1145/3570991.3571004. URL https://doi.org/10.1145/3570991.3571004.

Moritz Diehl. *Optimization Algorithms for Model Predictive Control*, pages 1–11. Springer London, London, 2013. ISBN 978-1-4471-5102-9. doi: 10.1007/978-1-4471-5102-9_9-1. URL https://doi.org/10.1007/978-1-4471-5102-9_9-1.

Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation, 2015. URL https://arxiv.org/abs/1410.8516.

Jiayuan Dong, Christian Jacobsen, Mehdi Khalloufi, Maryam Akram, Wanjiao Liu, Karthik Duraisamy, and Xun Huan. Variational bayesian optimal experimental design with normalizing flows. *Computer Methods in Applied Mechanics and Engineering*, 433: 117457, 2025. ISSN 0045-7825. doi: https://doi.org/10.1016/j.cma.2024.117457. URL https://www.sciencedirect.com/science/article/pii/S0045782524007126.

Shian Du, Yihong Luo, Wei Chen, Jian Xu, and Delu Zeng. To-flow: Efficient continuous normalizing flows with temporal optimization adjoint with moving speed, 2022. URL https://arxiv.org/abs/2203.10335.

Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987. ISSN 0370-2693. doi: https://doi.org/10.1016/0370-2693(87)91197-X. URL https://www.sciencedirect.com/science/article/pii/037026938791197X.

J. Durbin and S. J. Koopman. Time series analysis of non-gaussian observations based on state space models from both classical and bayesian perspectives. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 62(1):3–56, 2025/10/09/ 2000. URL http://www.jstor.org/stable/2680676. Full publication date: 2000.

J. Durbin and Siem Jan Koopman. *Time series analysis by state space methods*. Oxford statistical science series; 38. Oxford University Press, Oxford, 2nd ed edition, 2012. ISBN 9780199641178; 019964117X.

Richard Durrett. *Probability: theory and examples*. Duxbury Press, Belmont, CA, second edition, 1996. ISBN 0-534-24318-5.

Jos Elfring, Elena Torta, and René van de Molengraft. Particle filters: A Hands-On tutorial. *Sensors (Basel)*, 21(2), jan 2021.

Márcio Ferreira and Michał Bejger. Conditional variational autoencoder inference of neutron star equation of state from astrophysical observations. *Physical Review D*, 111(2), January 2025. ISSN 2470-0029. doi: 10.1103/physrevd.111.023035. URL http://dx.doi.org/10.1103/PhysRevD.111.023035.

Michel Fliess and Cédric Join. Model-free control. *International Journal of Control*, 86 (12):2228–2252, December 2013. ISSN 1366-5820. doi: 10.1080/00207179.2013.810345. URL http://dx.doi.org/10.1080/00207179.2013.810345.

Andrew Y. K. Foong, Yingzhen Li, José Miguel Hernández-Lobato, and Richard E. Turner. 'in-between' uncertainty in bayesian neural networks, 2019. URL https://arxiv.org/abs/1906.11537.

Andrew Y. K. Foong, David R. Burt, Yingzhen Li, and Richard E. Turner. On the expressiveness of approximate inference in bayesian neural networks, 2020. URL https://arxiv.org/abs/1909.00719.

Meire Fortunato, Charles Blundell, and Oriol Vinyals. Bayesian recurrent neural networks, 2019. URL https://arxiv.org/abs/1704.02798.

Marco Fraccaro, Simon Kamronn, Ulrich Paquet, and Ole Winther. A disentangled recognition and nonlinear dynamics model for unsupervised learning, 2017. URL https://arxiv.org/abs/1710.05741.

Gideon Joseph Freund, Elad Sarafian, and Sarit Kraus. A coupled flow approach to imitation learning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10357–10372. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/freund23a.html.

Hunter Gabbard, Chris Messenger, Ik Siong Heng, Francesco Tonolini, and Roderick Murray-Smith. Bayesian parameter estimation using conditional variational autoencoders for gravitational-wave astronomy. *Nature Physics*, 18(1):112–117, Jan 2022. ISSN 1745-2481. doi: 10.1038/s41567-021-01425-7. URL https://doi.org/10.1038/s41567-021-01425-7.

F. Gabbiani and S.J. Cox. *Mathematics for Neuroscientists*. Elsevier science & technology books. Elsevier Science, 2010.

Carlos E. García, David M. Prett, and Manfred Morari. Model predictive control: Theory and practice—a survey. *Automatica*, 25(3):335–348, 1989. ISSN 0005-1098. doi: https://doi.org/10.1016/0005-1098(89)90002-2. URL https://www.sciencedirect.com/science/article/pii/0005109889900022.

M. Gavrilović, R. Petrović, and D. Šiljak. Adjoint method in the sensitivity analysis of optimal systems. *Journal of the Franklin Institute*, 276(1):26–38, 1963. ISSN 0016-0032. doi: https://doi.org/10.1016/0016-0032(63)90307-7. URL https://www.sciencedirect.com/science/article/pii/0016003263903077.

Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Akti Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman & Hall/CRC Texts in Statistical Science Series. CRC, Boca Raton, Florida, third edition, 2013. ISBN 9781439840955 1439840954. URL https://stat.columbia.edu/~gelman/book/.

Samuel Gershman and Noah Goodman. Amortized inference in probabilistic reasoning. In *Proceedings of the annual meeting of the cognitive science society*, volume 36, 2014.

Raj Ghugare and Benjamin Eysenbach. Normalizing flows are capable models for rl, 2025. URL https://arxiv.org/abs/2505.23527.

Laurent Girin, Simon Leglaive, Xiaoyu Bie, Julien Diard, Thomas Hueber, and Xavier Alameda-Pineda. Dynamical variational autoencoders: A comprehensive review. *Foundations and Trends® in Machine Learning*, 15(1–2):1–175, 2021. ISSN 1935-8245. doi: 10.1561/2200000089. URL http://dx.doi.org/10.1561/2200000089.

Ethan Goan and Clinton Fookes. *Bayesian Neural Networks: An Introduction and Survey*, page 45–87. Springer International Publishing, 2020. ISBN 9783030425531. doi: 10.1007/978-3-030-42553-1_3. URL http://dx.doi.org/10.1007/978-3-030-42553-1_3.

N.J. Gordon, D.J. Salmond, and A.F.M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing)*, 140:107–113, 1993. doi: 10.1049/ip-f-2.1993.0015. URL https://digital-library.theiet.org/doi/abs/10.1049/ip-f-2.1993.0015.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models, 2018. URL https://arxiv.org/abs/1810.01367.

Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, December 2017. ISSN 1361-6420. doi: 10.1088/1361-6420/aa9a90. URL http://dx.doi.org/10.1088/1361-6420/aa9a90.

Paul Hagemann, Johannes Hertrich, and Gabriele Steidl. Stochastic normalizing flows for inverse problems: A markov chains viewpoint. *SIAM/ASA Journal on Uncertainty Quantification*, 10(3):1162–1190, September 2022. ISSN 2166-2525. doi: 10.1137/21m1450604. URL http://dx.doi.org/10.1137/21M1450604.

Katsiaryna Haitsiukevich, Onur Poyraz, Pekka Marttinen, and Alexander Ilin. Diffusion models as probabilistic neural operators for recovering unobserved states of dynamical systems, 2024. URL https://arxiv.org/abs/2405.07097.

W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970. ISSN 0006-3444. doi: 10.1093/biomet/57.1.97. URL https://doi.org/10.1093/biomet/57.1.97.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

Tor Aksel N. Heirung, Joel A. Paulson, Jared O'Leary, and Ali Mesbah. Stochastic model predictive control — how does it work? *Computers & Chemical Engineering*, 114:158–170, 2018. ISSN 0098-1354. doi: https://doi.org/10.1016/j.compchemeng.2017.10.026. URL https://www.sciencedirect.com/science/article/pii/S0098135417303812. FOCAPO/CPC 2017.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

Mai P. Ho, Yang Song, Perminder S. Sachdev, Jiyang Jiang, and Wei Wen. An enhanced conditional variational autoencoder-based normative model for neuroimaging analysis. *bioRxiv*, 2025. doi: 10.1101/2025.01.05.631276. URL https://www.biorxiv.org/content/early/2025/01/05/2025.01.05.631276.

Liam Hodgkinson, Chris van der Heide, Fred Roosta, and Michael W. Mahoney. Stochastic continuous normalizing flows: training SDEs as ODEs. In Cassio de Campos and Marloes H. Maathuis, editors, *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pages 1130–1140. PMLR, 27–30 Jul 2021.

M.F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19(2): 433–450, 1990. doi: 10.1080/03610919008812866. URL https://doi.org/10.1080/03610919008812866.

Pavel Izmailov, Sharad Vikram, Matthew D. Hoffman, and Andrew Gordon Wilson. What are bayesian neural network posteriors really like?, 2021. URL https://arxiv.org/abs/2104.14421.

Kalvik Jakkala. Deep gaussian processes: A survey, 2021. URL https://arxiv.org/abs/2106.12135.

Michael I. Jordan, Zoubin Ghahramani, Tommi S. Jaakkola, and Lawrence K. Saul. *An Introduction to Variational Methods for Graphical Models*. Springer Netherlands, Dordrecht, 1998. ISBN 978-94-011-5014-9. doi: 10.1007/978-94-011-5014-9_5. URL https://doi.org/10.1007/978-94-011-5014-9_5.

R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960. ISSN 0021-9223. doi: 10.1115/1.3662552. URL https://doi.org/10.1115/1.3662552.

Shahbaz Abdul Khader, Hang Yin, Pietro Falco, and Danica Kragic. Learning stable normalizing-flow control for robotic manipulation. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1644–1650, 2021. doi: 10.1109/ICRA48506.2021.9562071.

Grigory Khromov and Sidak Pal Singh. Some fundamental aspects about lipschitz continuity of neural networks, 2024. URL https://arxiv.org/abs/2302.10886.

Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions, 2018. URL https://arxiv.org/abs/1807.03039.

Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019. ISSN 1935-8245. doi: 10.1561/2200000056. URL http://dx.doi.org/10.1561/2200000056.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL https://arxiv.org/abs/1312.6114.

Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing flows: An introduction and review of current methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, November 2021. ISSN 1939-3539. doi: 10.1109/tpami.2020.2992934. URL http://dx.doi.org/10.1109/TPAMI.2020.2992934.

Lingkai Kong, Haichuan Wang, Tonghan Wang, Guojun Xiong, and Milind Tambe. Composite flow matching for reinforcement learning with shifted-dynamics data, 2025. URL https://arxiv.org/abs/2505.23062.

Jakob Kruse, Gianluca Detommaso, Ullrich Köthe, and Robert Scheichl. Hint: Hierarchical invertible neural transport for density estimation and bayesian inference, 2021. URL https://arxiv.org/abs/1905.10687.

Šimon Kucharský and Paul Christian Bürkner. Amortized bayesian cognitive modeling with bayesflow. *PsyArXiv*, May 2025. doi: 10.31234/osf.io/34k6q_v1. URL http://dx.doi.org/10.31234/osf.io/34k6q_v1.

S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 00034851. URL http://www.jstor.org/stable/2236703.

Mason Lee and Siddharth Diwan. Improving probabilistic model based reinforcement learning control in chaotic environments with conditional normalizing flows, 2023. URL https://api.semanticscholar.org/CorpusID:260067705.

Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations, 2020.

Yong Li, Sushanta K. Mallick, Nianling Wang, Jun Yu, and Tao Zeng. Deviance information criterion for bayesian model selection: Theoretical justification and applications. *Journal of Econometrics*, page 105978, 2025. ISSN 0304-4076. doi: https://doi.org/10.1016/j.jeconom.2025.105978. URL https://www.sciencedirect.com/science/article/pii/S0304407625000326.

Jiahe Lin and George Michailidis. Deep learning-based approaches for state space models: A selective review, 2024. URL https://arxiv.org/abs/2412.11211.

Zhidi Lin, Ying Li, Feng Yin, Juan Maroñas, and Alexandre H. Thiéry. Efficient transformed gaussian process state-space models for non-stationary high-dimensional dynamical systems, 2025. URL https://arxiv.org/abs/2503.18309.

Simon Kristoffersson Lind, Jialong Li, Maj Stenmark, and Volker Krüger. Normalizing flows are capable visuomotor policy learning models, 2025. URL https://arxiv.org/abs/2509.21073.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL https://arxiv.org/abs/2210.02747.

Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations, 2020. URL https://arxiv.org/abs/1710.10121.

Lei Lv, Yunfei Li, Yu Luo, Fuchun Sun, Tao Kong, Jiafeng Xu, and Xiao Ma. Flow-based policy for online reinforcement learning, 2025. URL https://arxiv.org/abs/2506.12811.

Xiaobai Ma, Jayesh K. Gupta, and Mykel J. Kochenderfer. Normalizing flow model for policy representation in continuous action multi-agent systems. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '20, page 1916–1918, Richland, SC, 2020. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9781450375184.

David J.C MacKay. Bayesian neural networks and density networks. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 354(1):73–80, 1995. ISSN 0168-9002. doi: https://doi.org/10.1016/0168-9002(94)00931-7. URL https://www.sciencedirect.com/science/article/pii/0168900294009317. Proceedings of the Third Workshop on Neutron Scattering Data Analysis.

Charles C. Margossian and David M. Blei. Amortized variational inference: When and why?, 2024. URL https://arxiv.org/abs/2307.11018.

Jorge Martínez-Palomera, Joshua S. Bloom, and Ellianna S. Abrahams. Deep generative modeling of periodic variable stars using physical parameters, 2020. URL https://arxiv.org/abs/2005.07773.

Bogdan Mazoure, Thang Doan, Audrey Durand, Joelle Pineau, and R Devon Hjelm. Leveraging exploration in off-policy algorithms via normalizing flows. In Leslie Pack Kaelbling, Danica Kragic, and Komei Sugiura, editors, *Proceedings of the Conference on*

Robot Learning, volume 100 of *Proceedings of Machine Learning Research*, pages 430–444. PMLR, 30 Oct–01 Nov 2020. URL https://proceedings.mlr.press/v100/mazoure20a.html.

David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients, 2025. URL https://arxiv.org/abs/2507.21053.

Arya D. McCarthy, Xian Li, Jiatao Gu, and Ning Dong. Addressing posterior collapse with mutual information for improved variational neural machine translation. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8512–8525, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.753. URL https://aclanthology.org/2020.acl-main.753/.

Christos Merkatas and Simo Särkkä. System identification using autoregressive bayesian neural networks with nonparametric noise models. *J. Time Ser. Anal.*, 44(3):319–330, May 2023.

Ali Mesbah. Stochastic model predictive control: An overview and perspectives for future research. *IEEE Control Systems Magazine*, 36(6):30–44, 2016. doi: 10.1109/MCS.2016.2602087.

Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 06 1953. ISSN 0021-9606. doi: 10.1063/1.1699114. URL https://doi.org/10.1063/1.1699114.

Ibomoiye Domor Mienye and Theo G. Swart. Deep autoencoder neural networks: A comprehensive review and new perspectives. *Archives of Computational Methods in Engineering*, Mar 2025. ISSN 1886-1784. doi: 10.1007/s11831-025-10260-5. URL https://doi.org/10.1007/s11831-025-10260-5.

Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey, 2022. URL https://arxiv.org/abs/2006.16712.

Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL http://probml.github.io/book1.

Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL http://probml.github.io/book2.

Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do bayesian inference, 2024. URL https://arxiv.org/abs/2112.10510.

Mayank Nautiyal, Andrey Shternshis, Andreas Hellander, and Prashant Singh. Variational autoencoders for efficient simulation-based inference, 2024. URL https://arxiv.org/abs/2411.14511.

Radford M. Neal. *Priors for Infinite Networks*, pages 29–53. Springer New York, New York, NY, 1996. ISBN 978-1-4612-0745-0. doi: 10.1007/978-1-4612-0745-0_2. URL https://doi.org/10.1007/978-1-4612-0745-0_2.

Bernt Øksendal. *Stochastic Differential Equations*, pages 38–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 1985. ISBN 978-3-662-13050-6. doi: 10.1007/978-3-662-13050-6_5. URL https://doi.org/10.1007/978-3-662-13050-6_5.

Jose L. Paniagua and Jesús A. López. Nonlinear system identification using modified variational autoencoders. *Intelligent Systems with Applications*, 22:200344, 2024. ISSN 2667-3053. doi: https://doi.org/10.1016/j.iswa.2024.200344. URL https://www.sciencedirect.com/science/article/pii/S2667305324000206.

George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation, 2018. URL https://arxiv.org/abs/1705.07057.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference, 2021. URL https://arxiv.org/abs/1912.02762.

Young-Jin Park, Piyush M. Tagade, and Han-Lim Choi. Deep gaussian process-based bayesian inference for contaminant source localization. *IEEE Access*, 6:49432–49449, 2018. doi: 10.1109/ACCESS.2018.2867687.

Ivo Pasmans, Yumeng Chen, Tobias Sebastian Finn, Marc Bocquet, and Alberto Carrassi. Ensemble kalman filter in latent space using a variational autoencoder pair, 2025. URL https://arxiv.org/abs/2502.12987.

V. Peterka. Bayesian system identification. *Automatica*, 17(1):41–53, 1981. ISSN 0005-1098. doi: https://doi.org/10.1016/0005-1098(81)90083-2. URL https://www.sciencedirect.com/science/article/pii/0005109881900832.

Mohammad Pivezhandi and Abusayeed Saifullah. A distribution-aware flow-matching for generating unstructured data for few-shot reinforcement learning, 2025. URL https://arxiv.org/abs/2409.14178.

Thomas Power and Dmitry Berenson. Variational inference mpc using normalizing flows and out-of-distribution projection, 2022. URL https://arxiv.org/abs/2205.04667.

Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 1994. ISBN 978-0-47161977-2. doi: 10.1002/9780470316887. URL https://doi.org/10.1002/9780470316887.

Stefan Radev. Deep learning architectures for amortized bayesian inference in cognitive modeling, 2021. URL https://archiv.ub.uni-heidelberg.de/volltextserver/id/eprint/30807.

Stefan T. Radev, Ulf K. Mertens, Andreas Voss, Lynton Ardizzone, and Ullrich Köthe. Bayesflow: Learning complex stochastic models with invertible neural networks, 2020. URL https://arxiv.org/abs/2003.06281.

Hazhir Rahmandad, Ali Akhavan, and Mohammad S Jalali. Incorporating deep learning into system dynamics: Amortized bayesian inference for scalable likelihood-free parameter estimation. *Syst. Dyn. Rev.*, 41(1), January 2025.

Davide Martino Raimondo, Daniel Limon, Mircea Lazar, Lalo Magni, and Eduardo Ferná ndez Camacho. Min-max model predictive control of nonlinear systems: A unifying overview on stability. *European Journal of Control*, 15(1):5–21, 2009. ISSN 0947-3580. doi: https://doi.org/10.3166/ejc.15.5-21. URL https://www.sciencedirect.com/science/article/pii/S0947358009707034.

Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian processes for machine learning*. MIT Press, 2008.

James Rawlings, David Mayne, and Moritz Diehl. *Model predictive control: theory, computation and design*. Nob Hill Publishing, LLC, Santa Barbara, second edition. edition, 2020. ISBN 9780975937754.

Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows, 2016. URL https://arxiv.org/abs/1505.05770.

Janosh Riebesell, Kapil Sachdeva, Juan Emmanuel Johnson, François Rozet, AifuHan, Byeongkeun Ahn, He Wang, Hossein Pourbozorg, Hyounguk Shon, John Franklin Crenshaw, Kristian Hartikainen, Kyle Cranmer, Matthew Skiffington, Maxime Vandegar, Rafael Orozco, Sandeep Nagar, Thorsten Glüsenkamp, Vincent Stimper, and kleinicke. janosh/awesome-normalizing-flows: v1.0.0, 2023.

Finn Rietz, Erik Schaffernicht, Stefan Heinrich, and Johannes A. Stork. Towards interpretable reinforcement learning with constrained normalizing flow policies, 2024. URL https://arxiv.org/abs/2405.01198.

Geoffrey Roeder, Paul K. Grant, Andrew Phillips, Neil Dalchau, and Edward Meeds. Efficient amortised bayesian inference for hierarchical and nonlinear dynamical systems, 2019. URL https://arxiv.org/abs/1905.12090.

Quentin Rouxel, Clemente Donoso, Fei Chen, Serena Ivaldi, and Jean-Baptiste Mouret. Extremum flow matching for offline goal conditioned reinforcement learning, 2025. URL https://arxiv.org/abs/2505.19717.

Lars Ruthotto and Eldad Haber. Deep neural networks motivated by partial differential equations, 2018. URL https://arxiv.org/abs/1804.04272.

J. Schwartz. The formula for change in variables in a multiple integral. *The American Mathematical Monthly*, 61(2):81–85, 1954. ISSN 00029890, 19300972. URL http://www.jstor.org/stable/2307790.

Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: an engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5):1327–1349, Nov 2021. ISSN 1433-3015. doi: 10.1007/s00170-021-07682-3. URL https://doi.org/10.1007/s00170-021-07682-3.

Kumar Shridhar, Felix Laumann, and Marcus Liwicki. A comprehensive guide to bayesian convolutional neural network with variational inference, 2019. URL https://arxiv.org/abs/1901.02731.

Daniele Silvestro and Tobias Andermann. Prior choice affects ability of bayesian neural networks to identify unknowns, 2020. URL https://arxiv.org/abs/2005.04987.

Dan Simon. *Optimal state estimation: Kalman, H ∞, and nonlinear approaches*. John Wiley & Sons, 2006.

John Skilling. *The Eigenvalues of Mega-dimensional Matrices*, pages 455–466. Springer Netherlands, Dordrecht, 1989. ISBN 978-94-015-7860-8. doi: 10.1007/978-94-015-7860-8_48. URL https://doi.org/10.1007/978-94-015-7860-8_48.

Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf.

Shriyank Somvanshi, Md Monzurul Islam, Mahmuda Sultana Mimi, Sazzad Bin Bashar Polock, Gaurab Chhetri, and Subasish Das. From s4 to mamba: A comprehensive survey on structured state space models, 2025. URL https://arxiv.org/abs/2503.18970.

David J. Spiegelhalter, Nicola G. Best, Bradley P. Carlin, and Angelika Van Der Linde. Bayesian measures of model complexity and fit. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 64(4):583–639, 10 2002. ISSN 1369-7412. doi: 10.1111/1467-9868.00353. URL https://doi.org/10.1111/1467-9868.00353.

David J. Spiegelhalter, Nicola G. Best, Bradley P. Carlin, and Angelika van der Linde. The deviance information criterion: 12 years on. *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, 76(3):485–493, 2014. ISSN 13697412, 14679868. URL http://www.jstor.org/stable/24774528.

Hui Sun, He Wang, and Jibo He. Accelerating bayesian sampling for massive black hole binaries with prior constraints from conditional variational autoencoder. *Physical Review D*, 111(10), May 2025. ISSN 2470-0029. doi: 10.1103/8r8b-hckx. URL http://dx.doi.org/10.1103/8r8b-hckx.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In Bruce Porter and Raymond Mooney, editors, *Machine Learning Proceedings 1990*, pages 216–224. Morgan Kaufmann, San Francisco (CA), 1990. ISBN 978-1-55860-141-3. doi: https://doi.org/10.1016/B978-1-55860-141-3.50030-4. URL https://www.sciencedirect.com/science/article/pii/B9781558601413500304.

Simo Särkkä. The use of gaussian processes in system identification, 2019. URL https://arxiv.org/abs/1907.06066.

Denis Tarasov, Alexander Nikulin, Ilya Zisman, Albina Klepach, Nikita Lyubaykin, Andrei Polubarov, Alexander Derevyagin, and Vladislav Kurenkov. Nina: Normalizing flows in action. training vla models with normalizing flows, 2025. URL https://arxiv.org/abs/2508.16845.

Travis Tune, Kristina B. Kooiker, Jennifer Davis, Thomas Daniel, and Farid Moussavi-Harami. Bayesian estimation of muscle mechanisms and therapeutic targets using variational autoencoders. *Biophysical Journal*, 124(1):179–191, 2025. ISSN 0006-3495. doi: https://doi.org/10.1016/j.bpj.2024.11.3310. URL https://www.sciencedirect.com/science/article/pii/S0006349524022380.

Julen Urain, Michele Ginesi, Davide Tateo, and Jan Peters. Imitationflow: Learning deep stable stochastic dynamic systems by normalizing flows. In *2020 IEEE/RSJ*

International Conference on Intelligent Robots and Systems (IROS), pages 5231–5237, 2020. doi: 10.1109/IROS45743.2020.9341035.

Martin J Wainwright and Michael I Jordan. *Graphical Models, Exponential Families, and Variational Inference*. Now Publishers Inc., Hanover, MA, USA, 2008. ISBN 1601981848.

E.A. Wan and R. Van Der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*, pages 153–158, 2000. doi: 10.1109/ASSPCC.2000.882463.

Zhenglin Wan, Jingxuan Wu, Xingrui Yu, Chubin Zhang, Mingcong Lei, Bo An, and Ivor Tsang. Fm-irl: Flow-matching for reward modeling and policy regularization in reinforcement learning, 2025. URL `https://arxiv.org/abs/2510.09222`.

Xintong Wang, Xiaofei Guan, Ling Guo, and Hao Wu. Flow-based bayesian filtering for high-dimensional nonlinear stochastic dynamical systems, 2025. URL `https://arxiv.org/abs/2502.16232`.

Yixin Wang, David M. Blei, and John P. Cunningham. Posterior collapse and latent variable non-identifiability, 2023. URL `https://arxiv.org/abs/2301.00537`.

Patrick Nadeem Ward, Ariella Smofsky, and Avishek Joey Bose. Improving exploration in soft-actor-critic with normalizing flows policies, 2019. URL `https://arxiv.org/abs/1906.02771`.

Junjie Wen, Minjie Zhu, Yichen Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Chengmeng Li, Xiaoyu Liu, Yaxin Peng, Chaomin Shen, and Feifei Feng. Diffusion-vla: Generalizable and interpretable robot foundation model via self-generated reasoning, 2025. URL `https://arxiv.org/abs/2412.03293`.

Christina Winkler, Daniel Worrall, Emiel Hoogeboom, and Max Welling. Learning likelihoods with conditional normalizing flows, 2023. URL `https://arxiv.org/abs/1912.00042`.

Hao Wu, Jonas Köhler, and Frank Noé. Stochastic normalizing flows, 2020. URL `https://arxiv.org/abs/2002.06707`.

Sheng Xu and Guiliang Liu. Uncertainty-aware constraint inference in inverse constrained reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=ILYjDvUM6U`.

Takahiro S. Yamamoto and Takahiro Tanaka. Use of conditional variational auto encoder to analyze ringdown gravitational waves, 2 2020.

Shi Yan, Yan Liang, Le Zheng, Mingyang Fan, Xiaoxu Wang, and Binglu Wang. Explainable gated bayesian recurrent neural network for non-markov state estimation, 2024. URL https://arxiv.org/abs/2310.17187.

Liu Yang, Xuhui Meng, and George Em Karniadakis. B-pinns: Bayesian physics-informed neural networks for forward and inverse pde problems with noisy data. *Journal of Computational Physics*, 425:109913, 2021. ISSN 0021-9991. doi: https://doi.org/10.1016/j.jcp.2020.109913. URL https://www.sciencedirect.com/science/article/pii/S0021999120306872.

Hang Yin, Christos K. Verginis, and Danica Kragic. Consensus-based normalizing-flow control: A case study in learning dual-arm coordination. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10417–10424, 2022. doi: 10.1109/IROS47612.2022.9981827.

Jice Zeng, Michael D. Todd, and Zhen Hu. Probabilistic damage detection using a new likelihood-free bayesian inference method. *Journal of Civil Structural Health Monitoring*, 13(2):319–341, Mar 2023. ISSN 2190-5479. doi: 10.1007/s13349-022-00638-5. URL https://doi.org/10.1007/s13349-022-00638-5.

Qinglun Zhang, Zhen Liu, Haoqiang Fan, Guanghui Liu, Bing Zeng, and Shuaicheng Liu. Flowpolicy: Enabling fast and robust 3d flow-based policy via consistency flow matching for robot manipulation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(14):14754–14762, Apr. 2025a. doi: 10.1609/aaai.v39i14.33617. URL https://ojs.aaai.org/index.php/AAAI/article/view/33617.

Shiyuan Zhang, Weitong Zhang, and Quanquan Gu. Energy-weighted flow matching for offline reinforcement learning, 2025b. URL https://arxiv.org/abs/2503.04975.

Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. Reinflow: Fine-tuning flow matching policy with online reinforcement learning, 2025c. URL https://arxiv.org/abs/2505.22094.

Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling, 2025d. URL https://arxiv.org/abs/2509.25756.

Zheng Zhang, Jinlin Zhu, Shuyu Zhang, and Furong Gao. Process monitoring using recurrent kalman variational auto-encoder for general complex dynamic processes. *Engineering Applications of Artificial Intelligence*, 123:106424, 2023. ISSN 0952-1976. doi: https://doi.org/10.1016/j.engappai.2023.106424. URL https://www.sciencedirect.com/science/article/pii/S0952197623006085.

Maksim Zhdanov, Lisa Randolph, Thomas Kluge, Motoaki Nakatsutsumi, Christian Gutt, Marina Ganeva, and Nico Hoffmann. Amortized bayesian inference of gisaxs data with normalizing flows, 2022. URL https://arxiv.org/abs/2210.01543.

Shan Zhong, Shutong Ding, He Diao, Xiangyu Wang, Kah Chan Teh, and Bei Peng. Flowcritic: Bridging value estimation with flow matching in reinforcement learning, 2025. URL https://arxiv.org/abs/2510.22686.

Feiqin Zhu, Dmitrii Torbunov, Zhongjing Jiang, Tianqiao Zhao, Amirthagunaraj Yogarathnam, Yihui Ren, and Meng Yue. Diffusion model-based parameter estimation in dynamic power systems, 2025. URL https://arxiv.org/abs/2411.10431.

Zhengbang Zhu, Hanye Zhao, Haoran He, Yichao Zhong, Shenyu Zhang, Haoquan Guo, Tingting Chen, and Weinan Zhang. Diffusion models for reinforcement learning: A survey, 2024. URL https://arxiv.org/abs/2311.01223.

Appendix

Acknowledgement

Eidesstattliche Versicherung

Statutory Declaration

1. Bei der eingereichten Dissertation zu dem Thema ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** handelt es sich um meine eigenständig erbrachte Leistung.
   I herewith formally declare that I have written the submitted dissertation ***Deep Bayesian Inference for Control: Normalizing Flow–Based Approaches to System Identification, State Estimation, and Optimal Control*** *independently.*
2. Ich habe nur die angegebenen Quellen und Hilfsmittel benutzt und mich keiner unzulässigen Hilfe Dritter bedient. Insbesondere habe ich wörtlich oder sinngemäß aus anderen Werken übernommene Inhalte als solche kenntlich gemacht.
   I did not use any third party support except for the quoted literature and other sources mentioned in the text. Content from other work, either literally or in content, has been declared as such.
3. Die Arbeit oder Teile davon habe ich bislang nicht an einer Hochschule des In- oder Auslands als Bestandteil einer Prüfungs- oder Qualifikationsleistung vorgelegt.
   The thesis has not been submitted to any examination body in this, or similar, form.
4. Die Richtigkeit der vorstehenden Erklärungen bestätige ich.
   I confirm the correctness of the aforementioned declarations.
5. Die Bedeutung der eidesstattlichen Versicherung und die strafrechtlichen Folgen einer unrichtigen oder unvollständigen eidesstattlichen Versicherung sind mir bekannt. Ich versichere an Eides statt, dass ich nach bestem Wissen die reine Wahrheit erklärt und nichts verschwiegen habe.
   I am aware of the legal consequences of this declaration. To the best of my knowledge I have told the pure truth and not concealed anything.

Augsburg, 02.01.2026 _____

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel

Information on the use of AI-based tools

Zur Dokumentation der verwendeten Hilfsmittel ist der schriftlichen Ausarbeitung ein besonderer Anhang hinzugefügt, der eine Liste und Beschreibung aller verwendeter KI-basierter Hilfsmittel enthält. Der besondere Anhang zur Dokumentation der verwendeten Hilfsmittel erfüllt folgende Kriterien:

To document the tools used, a special appendix has been added to the written report, which contains a list and description of all AI-based tools used. The special appendix to document the tools used meets the following criteria:

1. Auflistung der Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden.
   List of the goals for which the AI-based tools were used in this work.

2. Dokumentation der Verwendungsweise der KI-basierten Hilfsmittel.
   Documentation of how AI-based tools are used.

3. Nennung der Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen.
   Identification of the chapters and sections of this work in which AI-based tools were used to generate content.

Der Gebrauch dieser Hilfsmittel inklusive Art, Ziel und Umfang des Gebrauchs wurde mit meinem offiziellen Betreuer **Prof.Dr.Ing. Lars Mikelsons** abgesprochen.

*The use of these aids, including the type, purpose and extent of use, has been agreed with my official supervisor **Prof.Dr.Eng. Lars Mikelsons**.*

Mir ist bewusst, dass insbesondere der Versuch einer nicht dokumentierten Nutzung KI- basierter Hilfsmittel als Täuschungsversuch zu werten ist:

I am aware that in particular the attempt to use AI-based tools without documentation is to be regarded as an attempt at deception:

Gem. § 16 Abs. 2 der Promotionsordnung „Dr. med. / dent.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Kandidat/die Kandidatin bei einer Promotionsleistung getäuscht hat, so können einzelne oder alle Promotionsleistungen für ungültig erklärt werden. In schweren Fällen kann die Zulassung zum Promotionsverfahren zurückgenommen werden.“

According to § 16 paragraph 2 of the doctoral regulations for "Dr. med. / dent.": "If it emerges before the doctoral certificate is issued that the candidate has cheated in a doctoral achievement, individual or all doctoral achievements can be declared invalid. In serious cases, admission to the doctoral process can be withdrawn.

Und § 16 Abs. 2 der Promotionsordnung „Dr. sc. hum.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Doktorand / Doktorandin bei einer Promotionsleistung getäuscht hat, so kann der Promotionsausschuss diese Promotionsleistung oder alle bisher erbrachten Promotionsleistungen für ungültig erklären. In besonders schweren Fällen kann der Promotionsausschuss die Annahme als Doktorand / Doktorandin endgültig widerrufen.“

And § 16 paragraph 2 of the doctoral regulations “Dr. sc. hum.”: “If it emerges before the doctoral certificate is issued that the doctoral candidate has cheated in a doctoral achievement, the doctoral committee can declare this doctoral achievement or all doctoral achievements to date invalid. In particularly serious cases, the doctoral committee can permanently revoke the acceptance as a doctoral candidate.”

Augsburg, 02.01.2026 _____